

DSA Practice Notes

A Structured Notebook for Data Structures & Algorithms

Source Repository: github.com/manishrnl/Dsa-Practice-Problem

About the Author

Manish

B.Tech (2022 – 26) , Computer Science (AI & System Design)

Portfolio: manishrnl.in

GitHub Repository: github.com/manishrnl/Dsa-Practice-Problem

This notebook compiles practice problems, patterns, and notes across core DSA modules — Bit Manipulation, Number Theory, Recursion, Backtracking, Arrays, Sorting, Searching, Hashing, and Heaps — built while preparing for SDE / Java Backend interviews.

You tube Channel I follow for this course is

https://www.youtube.com/watch?v=N89PN_uyeIU&list=PLUcsbZa0qzu3yNzzAxgvSgRobdUUJvz7p&index=1

PLEASE OPEN THE PDF IN "ADOBE ACROBAT" ONLY since other apps does not fully responds to the hyperlink inside this documents. I had tested it many times and so came to this conclusion. Thanks

Index

Module	Topic	LeetCode / Reference	Page
00. Introduction	0.1 Introduction to DSA and Time Complexity	LC 509 - Fibonacci numbers	01 →
	0.2 Formulas on Recursion Tree , Master Theorem	LC 50 , LC 326	02 →
	0.3 Time Complexity via Recursion Tree method	LC 148: Sort List, LC 22	03 →
	0.4 Time Complexity via Master Theorem	LC 23: Merge k Sorted Lists, LC 240	05 →
	0.5 About 10⁸ Rule	LC 01 Two Sum	06 →
	0.6 Bit Masking – A Grace to Computing fields	LC 191, LC 136, LC 78	07 →
	0.7 (Blank pages for extra Notes)	—	10 →
01. Bit Manipulation	1.1 One Non-Repeating ,others appear twice	LC 136 - Single Number	11 →
	1.2 Two Non-Repeating ,others appear twice	LC 260 - Single Number III	13 →
	1.3 Find Single Element , Others Repeat Thrice	LC 137 - Single Number II	15 →
	1.4 (Blank pages for extra Bit Manipulation notes)	—	17 →
02. Number Theory	2.1 Fast / Modular Exponentiation (Calculate Power)	LC 50 - Pow(x, n)	19 →
	2.2 Catalan Numbers (BST count, valid parentheses)	LC 96 - Unique BST	21 →
	2.3 Factorial (Iterative / Recursive)	GfG - Factorial of a Number	23 →
	2.4 GCD / HCF of 2 Numbers (Euclidean Algorithm)	LC 1979 - Find GCD of Array	24 →
	2.5 Inclusion-Exclusion Principle	GfG - Incl-Exc Principle	25 →
	2.6 Palindrome Check (Number)	LC 9 - Palindrome Number	27 →
	2.7 Pigeonhole Principle Examples	GfG - Pigeonhole Principle	28 →
	2.8 Prime Numbers (Sieve of Eratosthenes)	LC 204 - Count Primes	30 →
	2.9 Count Trailing Zeroes in Factorial	LC 172 - Factorial Trailing 0's	31 →
	2.10 (Blank pages for extra Number Theory notes)	—	32 →
03. Recursion	3.1 Josephus Problem	GfG - Josephus Problem	33 →
	3.2 Permutations of an Array / String	LC 46 - Permutations	34 →
	3.3 Power Set (Subsets via Recursion)	LC 78 - Subsets	35 →
	3.4 String Palindrome (Recursive Check)	LC 125 - Valid Palindrome	36 →
	3.5 Unique Paths (Recursive Grid Traversal)	LC 62 - Unique Paths	37 →
	3.6 (Blank pages for extra Recursion notes)	—	38 →
04. Backtracking	4.1 N-Queens Problem	LC 51 - N-Queens	39 →
	4.2 Sudoku Solver	LC 37 - Sudoku Solver	41 →
	4.3 (Blank pages for extra Backtracking notes)	—	43 →

Module	Topic	LeetCode / Reference	Page
05. Array	5.1 Majority Element (count > N/2)	LC 169 - Majority Element	44 →
	5.2 Kadane's Algorithm (Maximum Subarray Sum)	LC 53 - Maximum Subarray	45 →
	5.3 Trapping Rainwater Problem	LC 42 - Trapping Rain Water	47 →
	5.4 Best Time to Buy and Sell Stock	LC 121 - Buy & Sell Stock	49 →
	5.5 (Blank pages for extra Array notes)	—	51 →
06. Sorting	6.1 Bubble Sort	LC 912 - Sort an Array	52 →
	6.2 Insertion Sort	LC 147 - Insertion Sort List	54 →
	6.3 Selection Sort	LC 912 - Sort an Array	55 →
	6.4 Merge Sort	LC 148 - Merge Sort	57 →
	6.5 Quick Sort	LC 912 - Sort an Array	60 →
	6.6 (Blank pages for extra Sorting notes)	—	62 →
07. Searching	7.1 Binary Search	LC 704 - Binary Search	63 →
	7.2 Allocate min no. of Pages (Google Interview)	LC 410 - Split Array Largest Sum	65 →
	7.3 Linear Search	GfG - Linear Search	67 →
	7.4 Search in an Infinite / Unbounded Sorted Array	LC 702 - Search in Unknown Size	68 →
	7.5 Search in Sorted and Rotated Array	LC 33 - Search in Rotated Sorted Array	70 →
	7.6 (Blank pages for extra Searching notes)	—	72 →
08. Hashing	8.1 Count Distinct Elements per Window of Size K	Count Distinct Elements per Window	73 →
	8.2 Find All Distinct Elements in an Array	LC 1207 - Unique Number Occurrences	75 →
	8.3 Intersection of Two Arrays	LC 217 - Contains Duplicate	76 →
		LC 349 - Intersection of Two Arrays	
	8.4 Subarray with Given Sum	LC 560 - Subarray Sum Equals K	77 →
	8.5 Union Two Arrays + Hashing Reports	GfG - Union of Two Arrays	79 →
		LC 706 - Design HashMap	
	8.6 (Blank pages for extra Hashing notes)	—	80 →
09. Heap (Priority Queue)	9.1 Min Heap Implementation	LC 215 - Kth Largest Elements	81 →
	9.2 Max Heap Implementation + Heap Reports	LC 1046 - Last Stone Weight	84 →
		LC 23 - Merge k Sorted Lists	
	9.3 (Blank pages for extra Heap Notes)	—	87 →

0.1 Introduction to DSA and Time Complexity

[← Back to Index](#)

Time taken by an algorithm as a function of the length of time input.

Time Complexity

Ex: $f(n) = 3n^2 + 5n \rightarrow O(n^2 + 1) \approx O(n^2)$

$f(n) = n + \log n \rightarrow O(n)$

$f(n) = 3n^3 + 4n^5 \rightarrow O(n^5)$

$f(n) = \text{loop} \rightarrow O(1)$

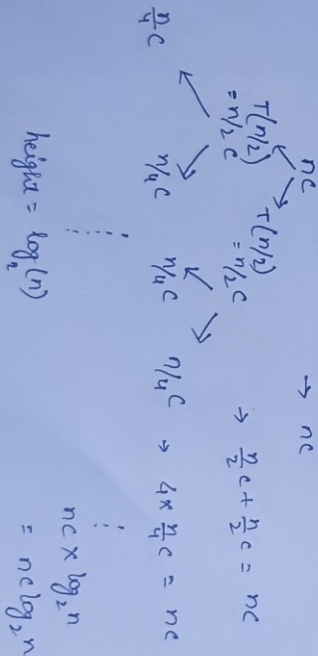
$\theta(f(n)) = \frac{n}{4}, \frac{n+100}{4}, 5n+3\log n \rightarrow O(n)$

$f(n) = \frac{n^2}{10}, \frac{n^2+200}{8}, 2n^n + 5\log n \rightarrow O(n^2)$

Q. find Time complexity for below program:

```
fun(n)
if (n==0) return 1; 1;
for (int i=0; i<n; i++) {
    fun(n/2); fun(n/2); }
    } c*xn = nc
```

Soln: $T(n) = 2T(n/2) + nc$
 $= 2[2T(n/4) + nc] + nc$
 $= 4T(n/4) + 3nc$



$= O(nc \log_2 n)$
 $= O(n \log n)$

Q. Algorithm A & B have worst-case running time of $O(n)$ & $O(\log n)$, respectively. Therefore, algorithm B always runs faster than Algorithm (A). True or false. [Ans - false]

Exp: $\rightarrow$ Big O notation describes asymptotic growth (how the time scales as n approaches ∞), not the absolute execution time for every possible input.

$\rightarrow$ Big O ignores constants & co-efficients.

eg: suppose Alg A = $10(n) \Rightarrow O(n)$

Alg B = $10000000 \log_2 n \Rightarrow O(\log n)$

But for smaller input size say $n=2$; A will finish in $10 \times 2 = 20$ unit of time while $10000000 \log_2 2$ will finish in 10000000 unit of time hence here A is faster despite being $O(n)$ over $O(\log n)$

$\rightarrow$ Also above Big O was for worst case, it might be possible that in best case $A = O(1)$ & $B = O(\log n)$ so in best case A might be faster. Nothing definite can be said Hence above statement is FALSE

0.2 Formulas on Recursion Tree , Master Theorem

[← Back to Index](#)

① LOG LAWS

Rule	Formula
Product	$\log(ab) = \log a + \log b$
Quotient	$\log(a/b) = \log a - \log b$
Power	$\log(a^n) = n \cdot \log a$
Base Change	$\log_b(a) = \log_c(a) / \log_c(b)$
Base Change (ln)	$\log_b(a) = \ln(a) / \ln(b)$
Base Change ($\log_{10}$)	$\log_b(a) = \log(a) / \log(b)$
Reciprocal	$\log_b(a) = 1 / \log_a(b)$
Identity	$\log_b(b) = 1 ; \log_b(1) = 0$
Inverse	$b^{\log_b(x)} = x$
Useful	$\log_2(n) = \log(n)/\log(2) \approx 3.32 \cdot \log_{10}(n)$

④ MASTER THEOREM

Recurrence: **$T(n) = a \cdot T(n/b) + f(n)$** , $a \geq 1$, $b > 1$

Compare: $f(n)$ vs $n^{\log_b a}$

Regularity condition: $\exists c < 1$ and large n such that $a \cdot f(n/b) \leq c \cdot f(n)$

Case	Condition	Result
Case 1	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$
Case 2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \cdot \log n)$
Case 3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ + regularity: $a \cdot f(n/b) \leq c \cdot f(n)$	$T(n) = \Theta(f(n))$

⑤ COMMON RECURRENCES (Master Theorem)

Recurrence	Case	Time
$T(n) = 2T(n/2) + O(n)$	Case 2	$O(n \log n)$
$T(n) = 2T(n/2) + O(1)$	Case 1	$O(n)$
$T(n) = T(n/2) + O(1)$	Case 2	$O(\log n)$
$T(n) = T(n/2) + O(n)$	Case 3	$O(n)$
$T(n) = 4T(n/2) + O(n)$	Case 1	$O(n^2)$
$T(n) = 4T(n/2) + O(n^2)$	Case 2	$O(n^2 \log n)$
$T(n) = 8T(n/2) + O(n^2)$	Case 1	$O(n^3)$
$T(n) = 2T(n/2) + O(n \log n)$	Case 2 ext	$O(n \log^2 n)$

⑥ RECURSION TREE METHOD

Step	Formula / Action
Tree depth	$d = \log_b(n)$
Nodes at level i	a^i
Work at level i	$a^i \cdot f(n/b^i)$
Total levels	$\log_b(n) + 1$
Leaves	$a^{\log_b n} = n^{\log_b a}$
Total work	$\sum_{i=0}^{\log_b n} a^i \cdot f(n/b^i)$
If each level = cn	Total = $cn \cdot \log_b(n) \rightarrow$ Case 2
Geometric growth (ratio > 1)	Sum = $\Theta(\text{last level}) \rightarrow$ Case 3
Geometric decay (ratio < 1)	Sum = $\Theta(\text{first level}) \rightarrow$ Case 1

② ARITHMETIC PROGRESSION (AP)

Formula	Expression
n-th term	$a_n = a + (n-1)d$
Sum of n terms	$S_n = n/2 \cdot [2a + (n-1)d]$
Alt sum formula	$S_n = n/2 \cdot (a + l)$ where l = last term
Sum 1.. n	$1+2+..+n = n(n+1)/2$
Sum of squares	$1^2+2^2+..+n^2 = n(n+1)(2n+1)/6$
Sum of cubes	$(1+2+..+n)^3 = [n(n+1)/2]^2$
Infinite AP	Diverges (no finite sum unless $d=0$)

③ GEOMETRIC PROGRESSION (GP)

Formula	Expression
n-th term	$a_n = a \cdot r^{n-1}$
Sum ($r \neq 1$)	$S_n = a \cdot (r^n - 1) / (r - 1)$
Sum ($r < 1$)	$S_n = a \cdot (1 - r^n) / (1 - r)$
Infinite GP $ r < 1$	$S_\infty = a / (1 - r)$
Infinite GP $ r \geq 1$	Diverges
Product of n terms	$P = (a \cdot l)^{n/2}$ where l = last term

⑦ KEY IDENTITIES

Identity	Formula
Symmetry	$n^{\log_b a} = a^{\log_b n}$
Geometric series	$\sum_{i=0}^k r^i = (r^{k+1} - 1) / (r - 1)$
Harmonic series	$\sum_{i=1}^n 1/i \approx \ln(n) + \gamma$, where $\gamma \approx 0.577$
Stirling's approx	$\log(n!) \approx n \cdot \log n - n$
Log of factorial	$\log_2(n!) = \Theta(n \log n)$

0.3 Time Complexity via Recursion Tree method

← Back to Index

Recursion Tree Method.

(3)

This is a visual approach where we "unroll" the recursive calls into a tree structure. It is highly versatile & works for almost any recurrence.

→ How it works:

- 1) Identify the work: for a recurrence like $T(n) = 2T(n/2) + n$, the "non-root" recursive" work at its root is n
- 2) Branching → "2" indicates that each node branches into 2 children of each $(n/2)$
- 3) Summing levels → Calculate total work done at each level
- 4) Find the height of tree. (usually $\log n$ for divide & conquer)
- 5) Total cost → Sum the work across all levels.

2) Master Theorem -

$T(n) = aT(n/b) + f(n)$; $\begin{cases} a \geq 1 \text{ (no. of sub-problems)} \\ b > 1 \text{ (factor by which input reduces)} \end{cases}$
 $f(n)$ work done outside recursive call

It compares $f(n)$ with $n \log_b a$, the critical value

Case 1 → if $f(n) < n \log_b a$ then $T(n) = \Theta(n \log_b a)$

Case 2 → if $f(n) = n \log_b a$ then $T(n) = \Theta(n \log_b a \cdot \log n)$

Case 3 → if $f(n) > n \log_b a$ then $T(n) = \Theta(f(n))$

Sum of GP.

$$S_n = \frac{a(1-r^{n+1})}{1-r} ; r \neq 1 \&$$

$$S_n = \frac{a(r^{n+1}-1)}{r-1} ; r \neq 1 \& r > 1$$

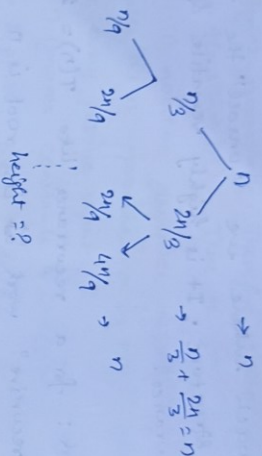
$$S_n = \frac{a}{1-r} \text{ for } r < 1 \text{ \& for } r > 1 \text{ series}$$

Sum of AP

$$S_n = \frac{n}{2} [2a + (n-1)d] \text{ or } \frac{n}{2} (a+d) \text{ or } \frac{n(n+1)}{2}$$

Q. Solve - $T(n) = T(n/3) + T(2n/3) + n$

Solⁿ



height = ?

To calculate height - tree will go upto height of $2n/3$

as we calculate when our function becomes $T(1)$

$n/3$ will become $T(1)$ very soon but $T(2n/3)$ will ends

at more time $\frac{2n}{3} > n/3$ hence we will calculate tree height for $2n/3$.

$n \cdot \left(\frac{2}{3}\right)^k = 1$; suppose at k level it becomes 1

$$n = \left(\frac{3}{2}\right)^k$$

$$\log_{3/2} n = k \log_{3/2} 3/2$$

hence tree height will go upto $\log_{3/2} n$

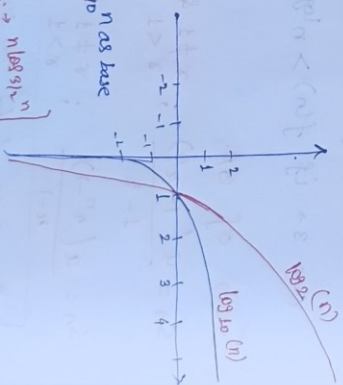
Now calculating time complexity

$$n + n + n \dots \log_{3/2} n \text{ times}$$

$$= n \times \log_{3/2} n$$

$$\Rightarrow \Theta(n \log_{3/2} n) \text{ "Ans"}$$

OR.



For standard way we usually take $\log_{10} n$ as base

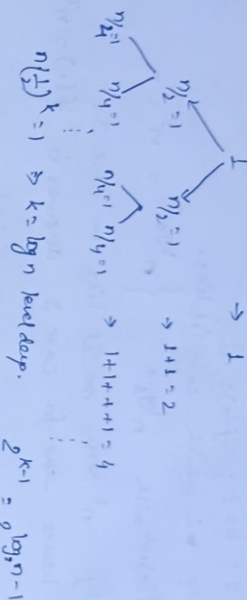
to converting $\log_{3/2} n$ as $\log_{10} n$

$$\log_a n = \frac{\log_b n}{\log_b a} \text{ [Base change formula]}$$

$$\Rightarrow \log_{3/2} n = \frac{\log_{10} n}{\log_{10} 3/2} = \log_{10} n \text{ constant}$$

$$\text{to } \log_{3/2} n \approx \log_{10} n \text{ hence Ans } \log(n \log n) \text{ "Ans"}$$

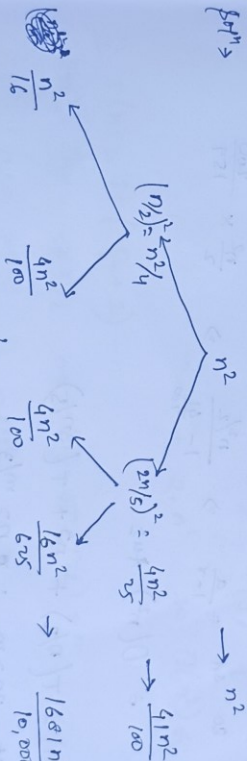
Q. $T(n) = 2T(n/2) + 1$



So sum for $\log n$ level deep is $\log n \times 2^{\log n} = n \log n$

Sum of $1 + 2 + 4 + 8 + 16 + \dots + 2^{\log n - 1}$
 Sum of GP where $a=1$; $r=2$; $n=2^{\log n}$
 $S_n = \frac{a(r^n - 1)}{r - 1} = \frac{1(2^n - 1)}{2 - 1} = 2^n - 1$
 $\Rightarrow \frac{2^{\log n} - 1}{2 - 1} = 2^{\log n} - 1 = n - 1$
 $\Rightarrow O(n)$ time

Q. $T(n) = T(n/2) + T(2n/5) + n^2$



Height: Max height for $n/2 \Rightarrow (n/2)^k = 1 \Rightarrow k = \log n$
 Height of tree $\approx \log n$
 $\Rightarrow$ last term for GP as $a=n^2$; $r=2$; $n=2^{\log n}$
 $a T_{last} = (n^2) \cdot 2^{\log n} = n^2 \log n$

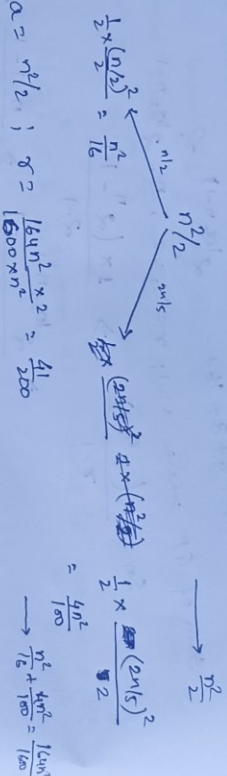
So last term $\approx n^2 \log n$
 Sum $\approx \frac{n^2}{1-r}$ for $r < 1$; infinite sum $\Rightarrow \frac{n^2}{1-1/2} = 2n^2 = O(n^2)$

Q. $T(n) = 2T(n/2) + n^2$ by Master Theorem

$a=2$; $b=2$; $f(n) = n^2$
 checking in which case it lies.
 comparing $f(n)$ with $n \log n$
 $\log n^2 = n^1 = n$

Clearly $n^2 > n$. so it lies in case 3.
 hence answer is $f(n) = n^2 \Rightarrow O(n^2)$

Q. $2T(n) = T(n/2) + T(2n/5) + n^2$
 $\Rightarrow T(n) = \frac{1}{2}T(n/2) + \frac{1}{2}T(2n/5) + \frac{n^2}{2}$



Tree height $= n(1/2)^k = 1$ or $k = \log n$
 $\Rightarrow S_n$ for $\infty = a(1-r^n)$ for ∞ ; $1-r^n \approx 1$
 So $S_n \approx \frac{a}{1-r} \Rightarrow \frac{n^2/2}{1-1/2} = n^2$
 $\Rightarrow O(n^2)$ time

Q. $2T(3n) = T(n/2) + n^2 + T(3n/5)$

Let $3n = m$; $n = m/3$
 $\Rightarrow 2T(m) = T(m/6) + (m/3)^2 + T(m/5)$
 $\Rightarrow T(m) = \frac{1}{2}T(m/6) + \frac{1}{2}(m/3)^2 + \frac{1}{2}T(m/5)$
 Now $n \geq m$ as variable didn't affect in final case.
 So $T(n) = \frac{1}{2}T(n/6) + \frac{1}{2} \times \frac{n^2}{9} + \frac{1}{2}T(n/5)$

$T(n) = \frac{1}{2}T(n/6) + \frac{n^2}{9} + \frac{1}{2}T(n/5)$

0.4 Time Complexity via Master Theorem

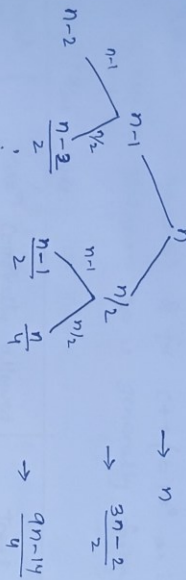
[← Back to Index](#)

$$\frac{1}{2} \times \frac{(n/2)^3}{5^4} = \frac{n^3}{2 \cdot 3 \cdot 2 \cdot 8} \xrightarrow{\text{for } n/5} \frac{1}{2} \times \frac{(n/5)^3}{5^4} = \frac{n^3}{1 \cdot 3 \cdot 5 \cdot 8} \rightarrow \frac{n^3}{\dots}$$

$$a = n^3/5^4 \quad ; \quad b = \dots$$

$$S_{\infty} = \frac{a}{1-b} \geq \frac{n^3/5^4}{1-\dots} \geq \frac{n^3}{\dots} = n^3 \text{ for } O(n^3) \text{ etc}$$

$$Q. T(n) = T(n-1) + T(n/2) + n$$



Depth = (n-1)

Converting to summation form $T(n) = \sum_{i=1}^n (i + T(i/2))$

$$\Rightarrow T(n) = \sum_{i=1}^n i + \sum_{i=1}^n T(i/2)$$

$$= n^2 + 2 \cdot \sum_{j=1}^{n/2} T(j) \quad \{ \text{let } j = i/2 \}$$

$$= n^2 + 2 \cdot n^2 \text{ so } O(n^2) \text{ etc}$$

$$Q. T(n) = 4T\left(\frac{n}{2} + 2\right) + n$$

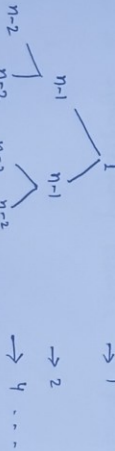
$$\text{Let } i = \frac{n}{2} + 2 \Rightarrow n/2 \text{ for } T(n) = 4T(n/2) + n$$

$$a = 4; b = 2; c = 1$$

$$\text{Cost} \rightarrow n \log_2 2^2 \Rightarrow n \log_2 4 \quad ; \text{ case 1 hence } n \log_2 4 = O(n \log_2 4)$$

$$Q. T(n) = 2T(n-1) + 1$$

Cost



Series are $1, 2, 4, \dots$ upto 2^n ; fill $(n-1)$ term.
So sum $= \frac{2^n(2^n - 1)}{2 - 1} = 2^n = O(2^n)$

0.5 About 10^8 Rule

← Back to Index

10^8 operations.

⑨

It helps whether an algorithm will pass within typical 1-sec time limit.

Suppose a CPU have clock speed of 3GHz ($= 3 \times 10^9$ cycle/sec). We use 10^8 not 10^9 because CPU is not 100% dedicated to our code only. CPU has multiple task running & via context switching it performs our task so we use as a safe zone for 10^8 op. Language like Java ($= 10^8$ due to JVM overhead) & Python ($= 10^7$ operation) & for C++ ($= 10^9$ as it has no JVM or similar platform so is generally faster)

Applying the rule.

Constraints (N)	Max ^m complexity allowed	Total operation (Appr.)
$N \leq 10$	$O(N!)$ or $O(2^N \cdot N^2)$	3.6×10^6
$N \leq 20$	$O(2^N)$	10^6
$N \leq 500$	$O(N^3)$	1.25×10^8 (Risky)
$N \leq 5,000$	$O(N^2)$	2.5×10^7
$N \leq 10^5$	$O(N \log_2 N)$ or $O(N)$	1.6×10^6
$N \leq 10^8$	$O(N)$	10^8 (Strict limit)
$N > 10^8$	$O(\log_2 N)$ or $O(1)$	Minimal

Language specific Naucses.

- C++ → It push 2×10^8 or even 5×10^8 operation/second
- Java → due to JVM overhead, pushes strictly to 10^8 /sec.
- Python → Significantly slower as pushes to 10^7 /sec.

How to use 10^8 Rule.

- 1) Read constraints say ($N=10^5$) & time limit = 1 sec obviously.
- 2) for 10^5 ; it should follow $O(N^2)$ or $O(N \log N)$
 for $O(N^2) = (10^5)^2 \gg 10^8$ ✗ Reject this Algo.
 for $O(N \log_2 N) = 10^5 \cdot \log_2 10^5 = 10^5 \times 17 \approx 10^6 \ll 10^8$ ✓ Accept
 so for $N=10^5$ use algo that has $O(N \log_2 N)$ as this will pass test within 1 sec.

Important:

- 1) Constant factor: $O(N)$ could mean $1 \cdot N$ or $10000N$ all have T.C = $O(N)$ but in real it affects a lot.
 so if constant is huge use $(\log_2 N)$ if possible.
- 2) I/O operations → Reading a massive i/p use Buffered Reader in Java.

[← Back to Index](#)

②

for subtraction

step-2 → Add 1)

2's complement

for say 64

x^2 2's complement

61 bits

$$\overbrace{000}^{61}$$

$$\underbrace{61.51}_{\text{61.51}}$$

Now Adding 1

0011

110
-
+.

we get

65 m bit
discard

1-

Left &

⑦ 12 >>

1100 >>

$\odot$
 $\bar{r}$
 $\wedge$
 $\wedge$

1100

Use of

Left

Stuft

Ex. 11

17 > 3

$17 \gg 4$

17 >> 5

like $17/9 = 4.25$

Bitwise

Integer

10

To check for Even & Odd.

BIT MASKING (13)

$\rightarrow$ if $(a \wedge 2 == 0)$ $\rightarrow$ if $(a \& 1 == 0)$
 "Even No."
 else "Odd No."
 else if $(a \& 1 == 1)$ "Even No."
 else if $(a \& 1 == 1)$ "Odd No."

And opⁿ with any for even no.

1000
 AND 0001
 0000
 $\rightarrow$ Performing 1000 & 1, last digit is 0 hence Even.

1000
 0001
 1001
 last digit is 1 for performing OR with 1000 with 1 hence odd.

Swap 2nd no. without 3rd variable.

Hint DO XOR 3 times & digits will be swapped
 int a = 5; int b = 7; swap value without 3rd var

$a \rightarrow 101$
 $b \rightarrow 111$
 $a \oplus b \rightarrow 010$
 $a \oplus (a \oplus b) \rightarrow 111$
 $b \oplus (a \oplus b) \rightarrow 101$
 step-1
 $a = a \oplus b = 7$
 $b = a \oplus b = 5$
 $a = a \oplus b = 7$

we first do $a \wedge b$ and result is 2. Now $a = 7$; $b = 7$
 now $a \wedge b = 2 \wedge 7 = 5$. This changes $b = 5$.
 Now we have $a = 7$; $b = 5$
 Again doing XOR for $a \wedge b = 2 \wedge 5 = 7$ & store in a
 thus it swaps $a = 7$, $b = 5$ in 3 XOR operation.

Bit Masking

1) Find i th Bit (whether 0 or 1) (Do & with 1)
 $n = 1001110101$ (say for $i = 5$ i.e., 6th & 7th)
 mask $\rightarrow 0000100000$ (Perform $n \& i$)

Now do & operation with n , mask

1000110101
 & 0000100000
 0000100000

if $(n \& \text{mask} == 1)$; means i th bit is 1
 if $(n \& \text{mask} == 0)$; means i th bit is 0

2) Set i th Bit (Do OR operation with 1)

$n = 1001110101$ (set $i = 4$ bit)
 mask = 0000010000
 $(n | \text{mask}) \rightarrow 1001111001$

To set 1 at i th bit do $(n | \text{mask})$ OR opⁿ with n , mask

3) Clear i th Bit

$n \rightarrow 1000110101$ (n original)
 $(1 \ll i) \rightarrow 0000010000$ ($1 \ll i$)
 $\sim(1 \ll i) \rightarrow 1111101111$ (mask)

Now do $n \& \text{mask}$

$n \rightarrow 1000110101$
 mask $\rightarrow 1111101111$
 1000110101
 1111101111
 1000101101
 $\rightarrow$ i th bit changes/clear from 1 to 0.

doing $n \& \sim(1 \ll i)$ to clear digit at i th place. ⁽¹⁵⁾
 in above example, digit at 5th place is set from 1 to 0
 Rest is unchanged.

X-OR - Problems.

Q1) Find only 1 non-repeating elements in an array where every elements repeats twice.

Ex $\rightarrow a = [5, 4, 1, 4, 3, 5, 1]$
 Here Only non-repeating is 3 others have pairs so ans is 3.

Property of X-OR.

$$\begin{array}{ll} 5 \wedge 5 = 0 & 3 \wedge 3 = 0 \\ 4 \wedge 4 = 0 & 1 \wedge 1 = 0 \\ 2 \wedge 0 = 2 & 9 \wedge 0 = 9 \end{array}$$

Solⁿ $\Rightarrow$ i) $0 \wedge 5 = 5$

ii) $5 \wedge 4 = 5 \wedge 4$

iii) $5 \wedge 4 \wedge 1 = 5 \wedge 4 \wedge 1$

iv) $5 \wedge 4 \wedge 1 \wedge 4 = 5 \wedge 1 \quad \{4 \wedge 4 = 0\}$

v) $5 \wedge 1 \wedge 3 = 5 \wedge 1 \wedge 3$

vi) $5 \wedge 1 \wedge 3 \wedge 5 = 1 \wedge 3 \quad \{5 \wedge 5 = 0\}$

$1 \wedge 3 \wedge 1 = 3 \quad \{1 \wedge 1 = 0\}$

final Ans = 3 which is non-repeating.

T.C = $O(n)$

Q. Find two non-repeating elements in an array where every elements repeats twice.

Ex $\rightarrow a = [5, 4, 1, 4, 3, 5, 1, 2]$ Ans $\rightarrow 3, 2$

~~public~~ public int[] find2UniqueXOR(int[] array)

int combinedXOR = 0;

for (int i : array) { combinedXOR ^= num; }

int rightmostBit = combinedXOR & -combinedXOR; Eg 6 & -6

int result[] = new int[2];

for (int num : array)

if ((num & rightmostBit) == 0) { result[0] ^= num; }

else result[1] ^= num;

return result;

for solⁿ via other method refer
github.com/manish0101/DSA-Practices
 Problem

1.1 Find the Single Non-Repeating Element (others appear twice)

Problem Statement

Find the only non-repeating element in an array where every other element repeats twice.

Problem Explanation - <https://www.geeksforgeeks.org/dsa/find-element-appears-array-every-element-appears-twice/>

Approach / Intuition

Code — Find the Single Non-Repeating Element (others appear twice)

```
import java.util.HashMap;
import java.util.Map;
public class Find1Unique {
    // General Approach T.C =  $O(n^2)$  , , Space Complexity =  $O(1)$ 
    public int find1UniqueGeneralApproach(int[] array) {
        for (int k : array) {
            int value = k, count = 0;
            for (int i : array)
                if (value == i) count++;
            if (count == 1) return value;
        }
        return -1;
    }

    // Hash Map Approach T.C =  $O(n)$  , Space Complexity =  $O(n)$ 
    public int find1UniqueHashMap(int[] array) {
        // 1. Create the "Tally Sheet" (HashMap) , The 1st data is the KEY 2nd data is the VALUE
        HashMap<Integer, Integer> results = new HashMap<>(array.length);
        for (int value : array)
            results.put(value, results.getOrDefault(value, 0) + 1);

        // 3. Look through the finished Tally Sheet
        // Map.Entry represents one "row" in your tally sheet (Key + Value)
        for (Map.Entry<Integer, Integer> entry : results.entrySet()) {
            // If the Value (the count) is exactly 1, we found our unique number!
            if (entry.getValue() == 1) {
                return entry.getKey(); // Return the Key (the original number)
            }
        }
    }
}
```

```

    // If we checked the whole map and found nothing
    return -1;
}

public int find1UniqueXOR(int[] array) {
    int result = 0;
    for (int i = 0; i < array.length; i++)
        result ^= array[i];
    return result;
}

public static void main(String[] args) {
    int[] array = {1, 2, 1, 3, 4, 6, 6, 7, 2, 3, 4, 5, 7};
    Find1Unique obj = new Find1Unique();

    // 1. General Approach
    long startGeneral = System.nanoTime();
    int resultGeneral = obj.find1UniqueGeneralApproach(array);
    long endGeneral = System.nanoTime();
    System.out.println("\n Results Via General Approach are : " + resultGeneral +
        " -> Execution Time: " + (endGeneral - startGeneral) + " ns");

    // 2. HashMap Approach
    long startHashMap = System.nanoTime();
    int resultHashMap = obj.find1UniqueHashMap(array);
    long endHashMap = System.nanoTime();
    System.out.println("\n Results via HashMap are : " + resultHashMap + " -> Execution" +
        " Time: " + (endHashMap - startHashMap) + " ns");

    // 3. XOR Approach
    long startXOR = System.nanoTime();
    int resultXOR = obj.find1UniqueXOR(array);
    long endXOR = System.nanoTime();
    System.out.println("\n Results via XOR are : " + resultXOR + " -> Execution Time: " + (endXOR - startXOR) + " ns");
}
}

```

Time Complexity

1.2 Find Two Non-Repeating Elements (others appear twice)

[← Back to Index](#)

Problem Statement

Find the two non-repeating elements in an array where every other element repeats twice.

<https://www.geeksforgeeks.org/dsa/find-two-non-repeating-elements-in-an-array-of-repeating-elements/>

Code — Find Two Non-Repeating Elements (others appear twice)

```
import java.util.*;

public class Find2Unique {

    public int[] find2UniqueGeneralApproach(int[] array) {
        int length = array.length;
        List<Integer> lists = new ArrayList<>();
        for (int value : array) {
            int count = 0;
            for (int i : array) {
                if (value == i) count++;
            }
            if (count == 1) lists.add(value);
        }
        int[] result = new int[lists.size()];
        for (int i = 0; i < lists.size(); i++)
            result[i] = lists.get(i);
        return result;
    }

    public int[] find2UniqueHashMap(int[] array) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int value : array) map.put(value, map.getOrDefault(value, 0) + 1);

        List<Integer> uniqueList = new ArrayList<>();
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            if (entry.getValue() == 1) uniqueList.add(entry.getKey());
        }
        int[] result = new int[uniqueList.size()];
        int index = 0;
        for (int num : uniqueList) result[index++] = num;
        return result;
    }

    public int[] find2UniqueXOR(int[] array) {
        // Step 1: XOR all elements.
        // Pairs cancel out. resultXOR will be (Unique1 ^ Unique2)
        int combinedXOR = 0;
        for (int num : array) combinedXOR ^= num;

        // Step 2: Isolate the rightmost "set bit" (the rightmost 1).
```

```

// This bit represents a position where Unique1 and Unique2 are DIFFERENT.
// Formula: x & -x gives the lowest bit that is 1.
int rightmostSetBit = combinedXOR & -combinedXOR;
int[] result = new int[2];

// Step 3: Divide numbers into two groups based on that specific bit.
for (int num : array) {
    // Group A: Numbers that have 0 at this bit position
    if ((num & rightmostSetBit) == 0) result[0] ^= num;
    // Group B: Numbers that have a 1 at this bit position
    else result[1] ^= num;
}
return result;
}

public static void main(String[] args) {
    int[] array = {1, 2, 3, 2, 1, 4};
    Find2Unique obj = new Find2Unique();
    long startGeneral = System.nanoTime();
    int[] resultGeneral = obj.find2UniqueGeneralApproach(array);
    long endGeneral = System.nanoTime();
    System.out.println("Result via General Approach is : " + Arrays.toString(resultGeneral) + " -> Execution Time : " + (endGeneral - startGeneral) + " ns");
    long startHashMap = System.nanoTime();
    int[] resultHashMap = obj.find2UniqueHashMap(array);
    long endHashMap = System.nanoTime();
    System.out.println("Result via Hash Map Approach is : " + Arrays.toString(resultHashMap) + " -> Execution Time : " + (endHashMap - startHashMap) + " ns");
    long startXOR = System.nanoTime();
    int[] resultXOR = obj.find2UniqueXOR(array);
    long endXOR = System.nanoTime();
    System.out.println("Result via XOR Approach is : " + Arrays.toString(resultXOR) + " -> Execution Time : " + (endXOR - startXOR) + " ns");
}
}

```

1.3 Find the Single Element When Others Repeat Thrice

[← Back to Index](#)

Problem Statement

Find the only non-repeating element in an array where every other element repeats thrice.
<https://www.geeksforgeeks.org/dsa/find-unique-element-element-occurs-k-times-except-one/>

Approach / Intuition

Code — Find the Single Element When Others Repeat Thrice

```
import java.util.*;
public class Find3Unique {

    public int find3UniqueHashMap(int[] array, int k) {
        HashMap<Integer, Integer> freqMap = new HashMap<>();

        // 1. Fill the frequency map
        for (int value : array)
            freqMap.put(value, freqMap.getOrDefault(value, 0) + 1);

        for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() != k) return entry.getKey();
        }
        return -1;
    }

    public int find3UniqueXOR(int[] array, int k) {

        int INT_SIZE = 32;
        int[] bitCounts = new int[INT_SIZE];

        // 2. Count the occurrences of '1' bits at each of the 32 positions
        for (int i = 0; i < INT_SIZE; i++) {
            for (int num : array) {
                // If the i-th bit of 'num' is set (1), increment the counter for that position
                if ((num & (1 << i)) != 0) {
                    bitCounts[i]++;
                }
            }
        }

        // Now consider all bits whose count is
        // not multiple of k to form the required
        // number.
        int res = 0;
    }
```

```

for (int i = 0; i < INT_SIZE; i++)
    res += (bitCounts[i] % k) * (1 << i);

// Before returning the res we need
// to check the occurrence of that
// unique element and divide it
res = res / (array.length % k);

return res;
}

public static void main(String[] args) {

    int[] array = {1, 2, 3, 2, 1, 3, 1, 2, 3, 8, 8};
    Find3Unique obj = new Find3Unique();

    long startHashMap = System.nanoTime();
    int resultHashMap = obj.find3UniqueHashMap(array, 3);
    long endHashMap = System.nanoTime();
    System.out.println("Result via Hash Map Approach is : " + (resultHashMap) + " -> Execution Time : " + (endHashMap -
startHashMap) + " ns");

    long startXOR = System.nanoTime();
    int resultXOR = obj.find3UniqueXOR(array, 3);
    long endXOR = System.nanoTime();
    System.out.println("Result via XOR Approach is : " + resultXOR + " -> Execution Time : " + (endXOR - startXOR) + "
ns");

}
}

```

Notes if Any :

2.1 Fast / Modular Exponentiation (Calculate Power)

Code — Fast / Modular Exponentiation (Calculate Power)

```
import java.math.BigInteger;

public class CalculatePower {
    public long calculatePower(long base, long power) {
        long result = 1;
        result = (long) Math.pow(base, power);
        System.out.printf("Power of %d ^ %d via Math.pow() is : ", base, power);
        return result;
    }

    /**
     * Optimized Power function using Binary Exponentiation.
     * Time Complexity: O(log power)
     * Space Complexity: O(1)
     */
    public long calculatePowerBitsOptimized(long base, long power) {
        System.out.printf("Power of %d ^ %d via Bits Calculation is : ", base, power);
        // Edge case: Any number to the power of 0 is 1
        if (power == 0) return 1;
        // Edge case: 0 to any positive power is 0
        if (base == 0) return 0;

        long result = 1;

        // Using a long for the base internally to prevent
        // immediate overflow during the squaring step
        long currentProduct = base;

        while (power > 0) {
            // If the least significant bit is 1, multiply result by currentProduct
            // (power & 1) is a faster way to check if power is odd
            if ((power & 1) != 0) {
                result *= currentProduct;
            }

            // Square the base for the next bit position
            currentProduct *= currentProduct;

            // Logical right shift to move to the next bit
            power = power >> 1;
        }

        return result;
    }
}
```

```

public long calculateFastPower(long base, long power, long n) {

    /* PROPERTY USED:
        (a + b) % n = (a % n + b % n) % n
        (a - b) % n = (a % n - b % n) % n
        (a * b) % n = (a % n * b % n) % n
        (a / b) % n = (a % n / b % n) % n

    */

    System.out.printf("Power of %d ^ %d with 10 ^ %d { %d } via Bits Calculation is : ",
        base, power, String.valueOf(n).length(), n);
    // Edge case: if n is 1, the result is always 0
    if(n == 1) return 0;
    long result = 1;

    // Initial mod to handle cases where base >= n
    base %= n;

    while (power > 0) {
        // If the current bit of 'power' is 1 (power is odd)
        // Multiply the result with the current base
        if((power & 1) != 0) {
            result = (result * base) % n;
        }

        // Square the base for the next bit position
        // Example: 3^1 -> 3^2 -> 3^4 -> 3^8
        base = (base * base) % n;

        // Right shift 'power' to process the next bit
        // Equivalent to power = power / 2
        power = power >> 1;
    }
    return result;
}

public BigInteger calculateTruePower(String base, long power) {
    BigInteger b = new BigInteger(base);
    BigInteger result = b.pow((int) power);
    System.out.printf("Power of %s ^ %d Via True Power is : ", base, power);
    return result;
}

public static void main(String[] args) {
    long base = 12333L, power = 266L;
    long n = 1000000000L;
    CalculatePower calculatePower = new CalculatePower();
    long startTime = System.currentTimeMillis();
    long resultMath = calculatePower.calculatePower(base, power);
    long endTime = System.currentTimeMillis();
}

```

```

System.out.println(resultMath + " -> Time taken to execute is : " + (endTime - startTime) + " ns");

long startBits = System.nanoTime();
long resultBits = calculatePower.calculatePowerBitsOptimized(base, power);
long endBits = System.nanoTime();
System.out.println(resultBits + " -> Time taken to execute is : " + (endBits - startBits) + " ns");

long startFastPower = System.nanoTime();
long resultFastPower = calculatePower.calculateFastPower(base, power, n);
long endFastPower = System.nanoTime();
System.out.println(resultFastPower + " -> Time taken to execute is : " + (endFastPower - startFastPower) + " ns");

long startTruePower = System.nanoTime();
BigInteger resultTruePower = calculatePower.calculateTruePower(String.valueOf(base),
    power);
long endTruePower = System.nanoTime();
System.out.println(resultTruePower + " -> Time taken to execute is : " + (endTruePower - startTruePower) + " ns");
}
}

```

Notes if Any :

Problem Statement

What are Catalan Numbers?

Catalan numbers are a sequence of natural numbers that occur in various counting problems, particularly those involving recursive structures.

The n -th Catalan number, denoted as C_n , counts things like:

- **Valid Parentheses:** The number of ways to arrange n pairs of $()$ so they are correctly nested.
- **Binary Search Trees:** The number of unique BSTs that can be formed with n nodes.
- **Mountain Ranges:** Ways to draw n upstrokes and n downstrokes that never stay below the horizon.

The sequence starts: **1, 1, 2, 5, 14, 42, 132...**

Approach / Intuition

The Mathematical Formula

While there is a recursive formula, the most efficient way to calculate C_n for a specific n is using the **Binomial Coefficient**:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Optimized Code (Linear Time)

This calculates C_n in $O(n)$ **time** and $O(1)$ **space**.

We use the iterative property: $C_{n+1} = \frac{2(2n+1)}{n+2} C_n$.

Code — Catalan Numbers (BST count, valid parentheses)

```
import java.math.BigInteger;

public class CatalanNumbers {

    // Time Complexity: O(n) , Space Complexity: O(1)
    public long findCatalanViaLong(int n) {
        long res = 1;

        // Using the formula: C(n) = C(n-1) * { (4n - 2) / (n + 1) }
        for (int i = 1; i <= n; i++) {
            // Multiply by (4i - 2)
            res = res * (4L * i - 2);
        }
    }
}
```

```

        // Divide by (i + 1)
        res = res / (i + 1);
    }
    return res;
}

public BigInteger findCatalanViaBigInteger(int n) {
    BigInteger result = BigInteger.ONE;
    for (int i = 1; i <= n; i++) {
        result = result.multiply(BigInteger.valueOf(4L * i - 2));
        result = result.divide(BigInteger.valueOf(i + 1));
    }
    return result;
}

public static void main(String[] args) {
    CatalanNumbers cn = new CatalanNumbers();
    int ng = 14, nb=4344;

    long start = System.nanoTime();
    long result = cn.findCatalanViaLong(ng);
    long end = System.nanoTime();

    System.out.println("The " + ng + "-th Catalan Number is: " + result);
    System.out.println("Length: " + String.valueOf(result).length() + " digits");
    System.out.println("Time: " + (end - start) + " ns");

    long start1 = System.nanoTime();
    BigInteger result1 = cn.findCatalanViaBigInteger(nb);
    long end1 = System.nanoTime();

    System.out.println("The " + nb + "-th Catalan Number is: " + result1);
    System.out.println("Length: " + result1.toString().length() + " digits");
    System.out.println("Time: " + (end1 - start1) + " ns");
}
}

```

Code — Factorial (Iterative / Recursive)

```
import java.math.BigInteger;

public class Factorial {
    public BigInteger findFactorialViaBigInteger(int n) {

        if (n <= 1) return BigInteger.ONE;

        return BigInteger.valueOf(n).multiply(findFactorialViaBigInteger(n - 1));

    }

    public long findFactorialGenerally(int n) {
        long factorial = 1;
        if (n <= 1) return 1;
        return (n * findFactorialGenerally(n - 1));

    }

    public static void main(String[] args) {
        Factorial factorial = new Factorial();
        long startTime = System.nanoTime();
        BigInteger result = factorial.findFactorialViaBigInteger(12);
        long endTime = System.nanoTime();
        int size = result.toString().length();

        System.out.printf(result + "\n Size of result is %d Digits long and -> Execution " +
            "time is %d ns ", size, (endTime - startTime));

        long startTime1 = System.nanoTime();
        long result1 = factorial.findFactorialGenerally(12);
        long endTime1 = System.nanoTime();
        long size1 = String.valueOf(result1).length();

        System.out.printf("\n Results are " + result1 + "\n Size of result is %d Digits long" +
            " and -> Execution time is %d ns ", size1, (endTime1 - startTime1));

    }
}
```

Notes if Any :

2.4 GCD / HCF of Two Numbers (Euclidean Algorithm)

[← Back to Index](#)

Code — GCD / HCF of Two Numbers (Euclidean Algorithm)

```
public class GCD {  
    public long greatestCommonDivision(int num1, int num2) {  
  
        if (num2 == 0) return num1;  
        if (num1 == 0) return num2;  
        return greatestCommonDivision(num1, num1 % num2);  
    }  
  
    public static void main(String[] args) {  
        GCD gcd = new GCD();  
        System.out.print("GCD of numbers are : ");  
        long startTime = System.nanoTime();  
        long result = gcd.greatestCommonDivision(0, 47);  
        long endTime = System.nanoTime();  
        System.out.println(result + " -> Execution time for function : " + (endTime - startTime) + " ns");  
    }  
}
```

Notes if Any :

Problem Statement

Inclusion-Exclusion is a counting technique used to find the size of the union of multiple sets. It helps to avoid the "double-counting" trap when sets overlap.

The "Party Guest" Rule

Imagine you are hosting a party and checking your guest list:

Step 1: You count everyone who likes **Pizza**.

Step 2: You count everyone who likes **Burgers**.

Step 3: You realize the people who like **BOTH** were counted twice! (Once in the Pizza group and once in the Burger group).

The Fix: You subtract the "Both" group once to get the true number of unique guests.

That is Inclusion-Exclusion.

Approach / Intuition

```
/**
 * @param index          Which prime we are currently looking at
 * @param currentDivisor  The product of the primes we've chosen so far
 * @param limit           The number we are checking up to (e.g., 100)
 * @param primes          Our list of divisors (e.g., {2, 3, 5})
 */
```

Instead of using confusing bitmasks, we can use a simple recursive "Decision Tree." For every number (like 2, 3, or 5), we have two choices: **Use it** or **Skip it**.

Code — Inclusion-Exclusion Principle

```
public class InclusionExclusion {
    public long countTotalDivisorsEfficient(int index, long currentDivisor, long limit,
                                           int[] primes) {
        // Base Case: If we've looked at all primes, stop.
        if (index == primes.length) return 0;

        // 1. Calculate for the current prime
        long combinedDivisor = currentDivisor * primes[index];

        // This is how many numbers are divisible by our "chosen" group
        long countWithThisPrime = limit / combinedDivisor;

        // 2. The Magic Logic:
        // We add the count of the current group,
        // then subtract the count of the NEXT group to fix the overlaps.
        return countWithThisPrime
            + countTotalDivisorsEfficient(index + 1, currentDivisor, limit, primes) // Option A: Skip this prime
            - countTotalDivisorsEfficient(index + 1, combinedDivisor, limit, primes); // Option B: Include this prime
    }

    public static void count_Total_Divisors_LongMethod(int n) {
        int count = 1;
    }
}
```

```

// Count the number of integers divisible by 2
for (int i = 2; i < n; i += 2) count++;

// Count the number of integers divisible by 3
for (int i = 3; i < n; i += 3) count++;

// Count the number of integers divisible by 5
for (int i = 5; i < n; i += 5) count++;

// Count the number of integers divisible by both 2 and 3
for (int i = 6; i < n; i += 6) count--;

// Count the number of integers divisible by both 2 and 5
for (int i = 10; i < n; i += 10) count--;

// Count the number of integers divisible by both 3 and 5
for (int i = 15; i < n; i += 15) count--;

// Count the number of integers divisible by 2, 3, and 5
for (int i = 30; i < n; i += 30) count++;

// Print the final count
System.out.println("Long Method Result: " + count);
}

public static void main(String[] args) {
    InclusionExclusion obj = new InclusionExclusion();
    int[] primes = {2, 3, 5};
    long limit = 1584565654L;

    // --- Timing Efficient Method ---
    long startEff = System.nanoTime();
    long resultEff = obj.countTotalDivisorsEfficient(0, 1, limit, primes);
    long endEff = System.nanoTime();
    System.out.println("Efficient Method Result: " + resultEff);
    System.out.println("Efficient Method Time: " + (endEff - startEff) + " ns");

    System.out.println("-----");

    // --- Timing Long Method ---
    long startLong = System.nanoTime();
    count_Total_Divisors_LongMethod((int) limit);
    long endLong = System.nanoTime();
    System.out.println("Long Method Time: " + (endLong - startLong) + " ns");
    // Final comparison
    double ratio = (double) (endLong - startLong) / (endEff - startEff);
    System.out.printf("\nThe Efficient method was %.2f times faster! \n", ratio);
}
}

```

2.6 Palindrome Check (Number)

[← Back to Index](#)

Code — Palindrome Check (Number)

```
public class Palindrome {

    public void palindrome(int n) {
        int result = 0;
        for (int i = n; i > 0; i /= 10) {
            int mod = i % 10;
            result = result * 10 + mod;
        }
        if (n == result)
            System.out.printf("Number %d is a palindrome", n);
        else System.out.printf("Number %d is not a palindrome", n);
    }

    public static void main(String[] args) {
        Palindrome test = new Palindrome();
        long startTime = System.nanoTime();
        test.palindrome(404);
        long endTime = System.nanoTime();
        System.out.print("-> Time taken for execute function is : " + (endTime - startTime) +
            " ns");
    }
}
```

Notes if Any :

Problem Statement

The Pigeonhole Principle (PHP) is a simple yet powerful concept in combinatorics. It states that:

If you have n items (pigeonholes) and m items (pigeons) to put into them, and $m > n$, then at least one pigeonhole must contain more than one pigeon.

Imagine you have 3 pigeonholes but 4 pigeons. No matter how hard you try to spread them out, one hole will end up with at least 2 pigeons.

Common Real-World Examples

- **Birthdays:** In a group of 367 people, at least two *must* share a birthday (since there are only 366 possible birthdays, including Feb 29).
- **Hair:** There are more people in New York City than there are hairs on a human head. Therefore, at least two people in NYC have the exact same number of hairs.
- **Arrays:** If you have an array of size N containing numbers from 1 to $N - 1$, at least one number must be a duplicate.

Coding Application: Finding a Duplicate

One of the most common ways we use PHP in coding is to find a duplicate in a constrained list.

Problem: You have an array of $n + 1$ integers where each integer is between 1 and n inclusive. Find the duplicate.

The "Fast" Way (Bit Manipulation / Frequency Array)

While we could use nested loops ($O(n^2)$), the Pigeonhole Principle tells us a duplicate **must** exist, so we can use a frequency array (the "holes") to catch it in $O(n)$ time.

Code — Pigeonhole Principle Examples

```
import java.util.ArrayList;
import java.util.List;

public class PigeonholeExample {
    public static int findDuplicate(int[] number) {
        boolean[] holes = new boolean[number.length];

        for (int num : number) {
            // If the 'hole' is already occupied, we found our duplicate!
            if (holes[num]) {
                return num;
            }
            // Otherwise, put the 'pigeon' in the hole
            holes[num] = true;
        }
        return -1; // Should not happen based on PHP
    }

    public static void totalDuplicate(int[] number) {
        List<Integer> totalDuplicates = new ArrayList<>();
        boolean[] holes = new boolean[number.length];
```

```

for (int value : number) {
    if (holes[value])
        totalDuplicates.add(value);
    holes[value] = true;
}
System.out.print("Total Duplicates value found are : ");
for (int value : totalDuplicates)
    System.out.print(value + " ");
}
public static void main(String[] args) {
    int[] pigeons = {1, 2, 3, 4, 2,3,4}; // 5 items, values 1-4
    System.out.println("Very 1st duplicate pigeon among all duplicates is: " + findDuplicate(pigeons));
    totalDuplicate(pigeons);
}
}

```

OUTPUT:

Very 1st duplicate pigeon among all duplicates is: 2

Total Duplicates value found are : 2 3 4

2.8 Prime Numbers (Sieve of Eratosthenes)

[← Back to Index](#)

Code — Prime Numbers (Sieve of Eratosthenes)

```
import java.util.Arrays;

public class PrimeNumbers {
    public boolean[] sieveOfEratoSthenes(int n) {

        boolean[] isPrime = new boolean[n + 1];
        Arrays.fill(isPrime, true);
        isPrime[0] = isPrime[1] = false; // Discarding index 0, 1 as we are interested in no.
        // from 2 to n . we keep position and index same for not getting confused ,
        // therefore initialised isPrime = new boolean[n+1]

        for (int i = 2; i * i <= n; i++) { // Iterate till squareRoot(n) times only as after
            // that all factors is checked
            for (int j = 2 * i; j <= n; j += i) { // Marking multiples of j to false so we
                // get true for only prime numbers once loops gets over
                isPrime[j] = false;
            }
        }
        return isPrime;
    }

    public static void main(String[] args) {
        PrimeNumbers p = new PrimeNumbers();
        long startTime = System.nanoTime();
        boolean[] result = p.sieveOfEratoSthenes(100);
        long endTime = System.nanoTime();
        System.out.print("Prime Numbers are : ");

        for (int i = 0; i < result.length; i++)
            if (result[i]) System.out.print(i + " ");
        System.out.print("-> Time taken to execute function is : " + (endTime - startTime) +
            " ns");
    }
}
```

OUTPUT:

Prime Numbers are : 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 -> Time taken to execute function is : 39800 ns

2.9 Count Trailing Zeroes in Factorial

[← Back to Index](#)

Problem Statement

Find number of trailing 0's in $n!$

Approach / Intuition

0 Occurs due to $2 * 5 = 10$, so count total no of 5 . that will give trailing zeroes.

REMEMBER : at 25 we get TWO 5's similarly at 125 , 625 , and so on , therefore we are checking loop from $i=5$ to $i \leq n$ at interval of $i*5$

Code — Count Trailing Zeroes in Factorial

```
public class Trailing_Zeroes {

    public int trailingZeroes(int n) {
        int result = 0;
        System.out.print("Trailing Zeroes in " + n + " ! is : ");
        for (int i = 5; i <= n; i *= 5) {
            result += n / i;
        }

        return result;
    }

    public static void main(String[] args) {

        Trailing_Zeroes test = new Trailing_Zeroes();
        long startTime = System.nanoTime();
        int result = test.trailingZeroes(87);
        long endTime = System.nanoTime();
        System.out.print(result + " -> Time Taken to compute is : " + (endTime - startTime) + " ns");

    }
}
```

OUTPUT:

Trailing Zeroes in 87 ! is : 20 -> Time Taken to compute is : 1886200 ns

Module 03: Recursion

3.1 Josephus Problem

[← Back to Index](#)

Problem Statement

This uses a Recursive Relation. If we know the survivor position for $(n-1)$ people, we can find the position for n people by shifting the index by 'k' and taking the modulo 'n'.

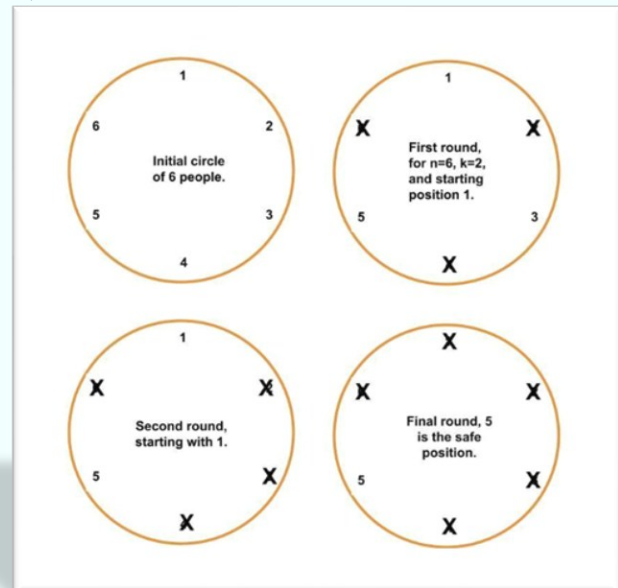
@param n Total number of people

@param k The step rate (every k-th person is eliminated)

@return The 0-indexed position of the survivor

Code — Josephus Problem

```
public static long josephProblem(int n, int k) {  
    // Base Case: If there is only 1 person, they are the survivor (at index 0)  
    if (n == 1) {  
        return 0;  
    }  
    /* * Recursive Step:  
    * 1. Solve the problem for n-1 people.  
    * 2. Add k to shift the starting point.  
    * 3. Use % n to wrap around the circle.  
    */  
    return (josephProblem(n - 1, k) + k) % n;  
}  
  
public static void main(String[] args) {  
    int n = 6;  
    int k = 2;  
  
    long survivorPosition = josephProblem(n, k);  
    System.out.println("The survivor in a circle of " + n + " people with a step of " + k + " starting from position 1 is at index: " +  
        survivorPosition);  
}
```



OUTPUT:

The survivor in a circle of 6 people with a step of 2 starting from position 1 is at index: 5

Notes if Any :

Problem Statement

```
/**
 * Problem: Print All Permutations
 * Logic: Use Backtracking. We swap the current character with every character appearing after it to create new arrangements.
 * After the recursive call, we swap back (backtrack) to restore the original string for the next iteration.
 * @param str The string to permute
 * @param startIndex The starting index of the window
 * @param endIndex The ending index of the window
 */
```

Code — Permutations of an Array / String

```
public class Permutations {
    public static void printAllPermutations(String str, int startIndex, int endIndex) {
        // Base Case: If the window has narrowed to a single character
        if (startIndex == endIndex) {
            System.out.print(str + " ");
            return;
        }
        for (int i = startIndex; i <= endIndex; i++) {
            // Step 1: Swap character at 'left' with character at 'i'
            str = swap(str, startIndex, i);

            // Step 2: Recurse for the rest of the string
            printAllPermutations(str, startIndex + 1, endIndex);

            // Step 3: Backtrack (Swap back to original)
            str = swap(str, startIndex, i);
        }
    }
    private static String swap(String str, int i, int j) {
        char[] charArray = str.toCharArray();
        char temp = charArray[i];
        charArray[i] = charArray[j];
        charArray[j] = temp;
        return String.valueOf(charArray);
    }
    public static void main(String[] args) {
        String s = "abcd";
        System.out.println("Permutations of " + s + ": \n");
        printAllPermutations(s, 0, s.length() - 1);
    }
}
```

3.3 Power Set (Subsets via Recursion)

[← Back to Index](#)

Problem Statement

- * *Problem: Print Power Set*
- * *Logic: For every character in the string, we have two choices:*
 - * *1. Include the character in the current subset.*
 - * *2. Exclude the character from the current subset.*
- * *This creates a binary recursion tree.*
- * *@param str The original string*
- * *@param index The current character we are considering*
- * *@param current The subset built so far*

Code — Power Set (Subsets via Recursion)

```
public class PowerSet {
    public static void printPowerSet(String str, int index, String current) {
        // Base Case: If we have considered all characters
        if(index == str.length()) {
            System.out.println "\"" + current + "\"");
            return;
        }

        // Choice 1: Include the current character
        printPowerSet(str, index + 1, current + str.charAt(index));

        // Choice 2: Exclude the current character
        printPowerSet(str, index + 1, current);
    }

    public static void main(String[] args) {
        String s = "abc";
        System.out.println("Power Set of " + s + ":");
        printPowerSet(s, 0, "");
    }
}
```

Notes if Any :

3.4 String Palindrome (Recursive Check)

[← Back to Index](#)

Code — String Palindrome (Recursive Check)

```
public class StringPalindrome {  
    // This matches your requested signature  
    public static boolean stringPalindrome(String str) {  
        // Base case: null or empty strings are technically palindromes  
        if (str == null || str.isEmpty()) return true;  
  
        // Call the helper method with the initial start and end positions  
        return stringPalindrome(str, 0, str.length() - 1);  
    }  
  
    // This is the recursive helper that uses your specific logic  
    private static boolean stringPalindrome(String str, int startIndex, int lastIndex) {  
        // Base case: If the pointers meet or cross, every character matched  
        if (startIndex >= lastIndex) return true;  
  
        char startChar = str.charAt(startIndex);  
        char endChar = str.charAt(lastIndex);  
  
        // If characters at current pointers don't match, it's not a palindrome  
        if (startChar != endChar) return false;  
  
        // Recursive call: move pointers inward  
        return stringPalindrome(str, startIndex + 1, lastIndex - 1);  
    }  
    public static void main(String[] args) {  
        String str = "Mams";  
        boolean result = stringPalindrome(str.toLowerCase());  
        String response = result ? "It is a Palindrome " : "It is not a Palindrome ";  
        System.out.println(response);  
    }  
}
```

Notes if Any :

3.5 Unique Paths (Recursive Grid Traversal)

[← Back to Index](#)

Problem Statement

- * *Logic: This is a recursive implementation of the "Pascal's Identity".*
- * *Every cell in the grid (except the first row and first column) can be reached from exactly two directions:*
 - * 1. From the Top (rows - 1)
 - * 2. From the Left (columns - 1)
- * * *Base Case: If you are in the first row (rows == 1) or first column (columns == 1), there is only 1 straight path to reach any cell in that line.*
- * * *Complexity: T.C: $O(2^{(n+m)})$, S.C : $O(n+m)$ - Due to the recursion stack.*

Code — Unique Paths (Recursive Grid Traversal)

```
public class UniquePaths {  
    public static long countUniquePaths(int rows, int columns) {  
        // If we are at the edge (first row or first column), only 1 way exists (straight line)  
        if (rows == 1 || columns == 1) return 1;  
        // Total ways = ways from the top cell + ways from the left cell  
        return countUniquePaths(rows - 1, columns) + countUniquePaths(rows, columns - 1);  
    }  
  
    // Optimized above code to  $O(n*m)$  version  
    public static long countPathsDP(int rows, int columns) {  
        long[][] dp = new long[rows][columns];  
  
        for (int i = 0; i < rows; i++) {  
            for (int j = 0; j < columns; j++) {  
                if (i == 0 || j == 0) dp[i][j] = 1; // Base case  
                else dp[i][j] = dp[i - 1][j] + dp[i][j - 1]; // Sum of top and left  
            }  
        }  
        return dp[rows - 1][columns - 1];  
    }  
  
    public static void main(String[] args) {  
        int rows = 12, columns = 15;  
        long startTime = System.nanoTime();    long result = countUniquePaths(rows, columns);  
        long endTime = System.nanoTime();  
        System.out.println("Results via Recursion Problem: " + result + " in " + (endTime - startTime) + " ns");  
        long startTime1 = System.nanoTime();    long result2 = countPathsDP(rows, columns);  
        long endTime1 = System.nanoTime();  
        System.out.println("Results via DP Approach : " + result2 + " in " + (endTime1 - startTime1) + " ns");  
    }  
}
```

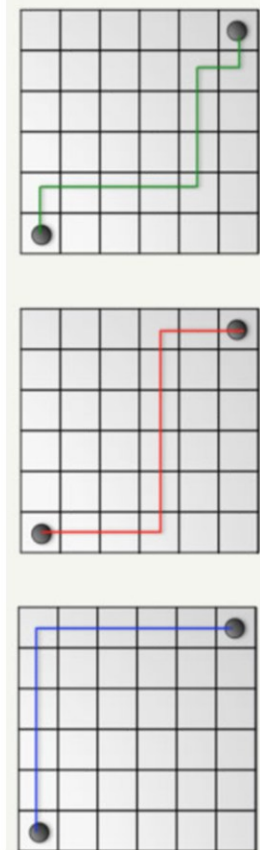


Figure 2: Three unique paths on a 6x6 grid

Module 04: Backtracking

4.1 N-Queens Problem

[← Back to Index](#)

Problem Statement

Given “n” Queen to be placed in a n x n chess board in such a way that no 2 queen can attack each other.

Approach / Intuition

Place Queens at any place starting from [0,0] now check if next queen can be placed in a next row such that it cant cancel each other . if they attacks , backtrack one step and recheck for next place.

```
/**
 * We only need to check 3 directions (Up, Up-Left, Up-Right)
 * because we haven't placed any queens below the current row yet.
 * Also bottom will always be empty , we are inserting from top .
 * so checking bottom doesn't make sense
 */
```

Code — N-Queens Problem

```
public class N_Queen {
    static int N = 25;
    public static boolean solve(int[][] board, int row) {
        // BASE CASE: If we reached the end, we found a solution!
        if (row == N) return true;

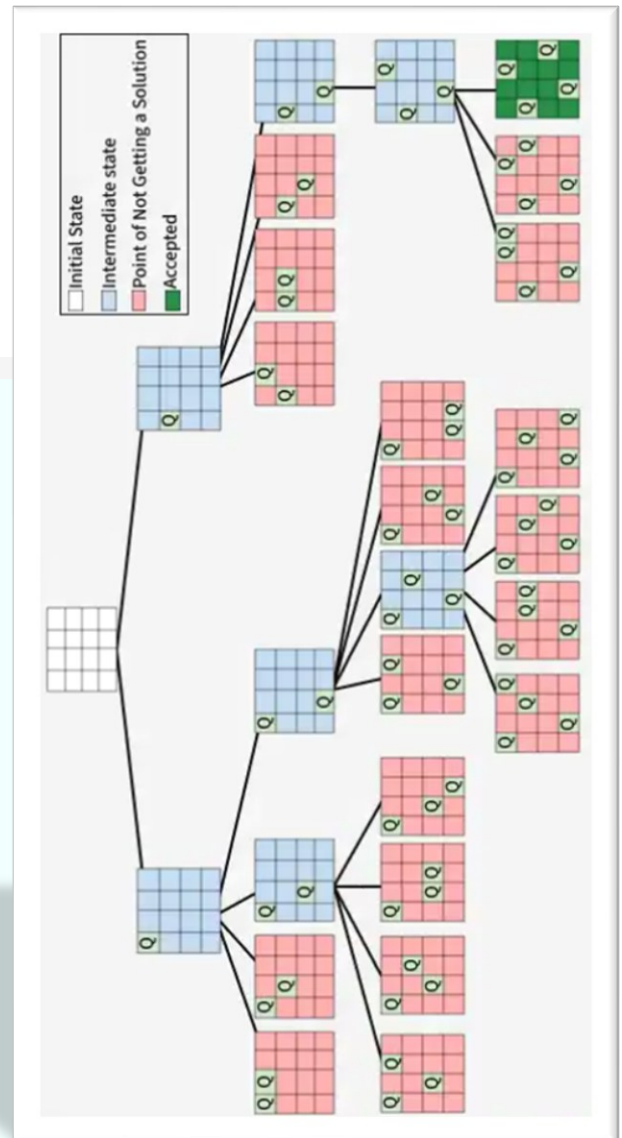
        for (int col = 0; col < N; col++) {
            if (isSafe(board, row, col)) {
                board[row][col] = 1; // Place Queen

                // Move to the next row
                if (solve(board, row + 1)) return true;

                // BACKTRACK: "Undo" the placement if it didn't work out
                board[row][col] = 0;
            }
        }
        return false;
    }

    private static boolean isSafe(int[][] board, int row, int col) {
        // We only need to check 3 directions (Up, Up-Left, Up-Right) because we haven't placed any queens below the current row yet. Also bottom will always be empty , we are inserting from top . so checking bottom doesn't make sense

        for (int i = 1; i <= row; i++) {
            // 1. Check Straight Up till (row,0)
```



```

    if (board[row - i][col] == 1) return false;

    // 2. Check Up-Left Diagonal till (row-i,col-i)
    if (col - i >= 0 && board[row - i][col - i] == 1) return false;

    // 3. Check Up-Right Diagonal till (row-i,col+i)
    if (col + i < N && board[row - i][col + i] == 1) return false;
}
return true;
}

public static void main(String[] args) {
    long startTime = System.nanoTime();
    int[][] board = new int[N][N];
    if (solve(board, 0)) {
        long endTime = System.nanoTime();
        System.out.printf(" Time taken to complete backTracking : %d ns\n", (endTime - startTime));
        printBoard(board);
    }
}

private static void printBoard(int[][] board) {
    for (int[] row : board) {
        for (int cell : row) {
            System.out.print(cell == 1 ? "Q " : ". ");
        }
        System.out.println();
    }
}
}

```

Notes if Any :

Problem Statement

This Sudoku solver is a perfect example of Recursive Backtracking . It essentially uses a "Brute Force" approach but with a brain—it drops a path the moment it violates a rule.

Approach / Intuition

Before the algorithm can try a number, it needs to know *where* to put it. Once it finds the very first digit = 0 , it breaks , goes inside another loop to check at **safeSearch(...)** , then if placing digits is successfully done, it again comes back here to find next 0 . This continues until all are filled and no 0's are left . Once a cell is picked, the code enters a loop from 1 to 9

The Validation Phase (isSafe): This is the "Rules of the Game" section. It checks three distinct constraints:

```
if(board[row][i]==num || board[i][col]==num) return false;
```

It scans the entire horizontal row and vertical column. If num exists there, it's an immediate "No."

The 3x3 Grid Check : *This is usually the most confusing part for developers. How do we know which 3x3 box we are in?*

The "Validator" Logic (Explained Meaningfully). The **isSafe(..)** function acts as a pre-condition validator. It treats the num as a candidate, checking its compatibility against the existing constraints of the board (Row, Column, and 3x3 Box) before any state change occurs. Because the candidate is not yet written into the matrix, the function effectively performs a collision check in a read-only state. If the candidate clears all three 'gatekeepers' without a conflict, the calling function is granted permission to 'commit' that value to the board. If a conflict is found, the candidate is rejected immediately, saving the algorithm from exploring an invalid path.

Code — Sudoku Solver

```
public class SudokuSolver {

    public static boolean solveSudoku(int[][] board) {
        int n = board.length;    int row = -1;    int col = -1;    boolean emptyLeft = false;
        // 1. Find the first empty cell (0)
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == 0) {
                    row = i;    col = j;    emptyLeft = true;    break;
                }
            }
        }
        if (emptyLeft) break;
    }
    // Base Case: No empty cells left, Sudoku is solved!
    if (!emptyLeft) return true;

    // 2. Try numbers 1 to 9
    for (int num = 1; num <= 9; num++) {
        if (isSafe(board, row, col, num)) {
```

```

        board[row][col] = num;
        if (solveSudoku(board)) return true;           // Move to next step
        board[row][col] = 0;                         // BACKTRACK: Undo the choice
    }
}
return false;
}

private static boolean isSafe(int[][] board, int row, int col, int num) {
    // Check Row and Column
    int rowLength = board.length;
    int colLength = board[0].length;
    for (int i = 0; i < rowLength; i++) {
        if (board[row][i] == num || board[i][col] == num) return false;
    }
    // Check 3x3 Sub-grid This math finds the "top-left" corner of the 3x3 box
    int rowStart = row - (row % 3);
    int colStart = col - (col % 3);

    for (int i = rowStart; i < rowStart + 3; i++) {
        for (int j = colStart; j < colStart + 3; j++) {
            if (board[i][j] == num) return false;
        }
    }
    return true;
}

public static void main(String[] args) {
    int[][] board = {
        {8, 3, 0, 1, 0, 0, 6, 0, 5},
        {0, 0, 0, 0, 0, 0, 8, 0},
        {0, 0, 0, 7, 0, 0, 9, 0, 0},
        {0, 5, 0, 0, 1, 7, 0, 0, 0},
        {0, 0, 3, 0, 0, 0, 2, 0, 0},
        {0, 0, 0, 3, 4, 0, 0, 1, 0},
        {0, 0, 4, 0, 0, 8, 0, 0, 0},
        {0, 9, 0, 0, 0, 0, 0, 0, 0},
        {3, 0, 2, 0, 0, 6, 0, 4, 7}
    };
    if (solveSudoku(board)) printBoard(board);
}

private static void printBoard(int[][] board) {
    for (int[] row : board) {
        for (int cell : row) System.out.print(cell + " ");
        System.out.println();
    }
}
}

```

8	3							

Notes if Any :

5.2 Kadane's Algorithm (Maximum Subarray Sum)

[← Back to Index](#)

Problem Statement

Finds the Maximum Subarray Sum using Kadane's Algorithm.

Approach / Intuition

THE LOGIC: This algorithm is an optimized "Greedy" approach that solves the problem in a single pass ($O(N)$). It operates on a simple principle: "A subarray should only continue as long as its sum remains positive. If the 'currentSum' becomes negative, it means the preceding elements are a 'burden' that would reduce the sum of any future subarray. Therefore, we 'reset' the currentSum to 0 and start a fresh subarray from the next element.

Code — Kadane's Algorithm (Maximum Subarray Sum)

```
public class KadaneAlgorithm {
    public static int max_Pair_Sum(int[] array) {
        // 'maxSum' acts as the global record holder. 'currentSum' tracks the current contiguous sequence.
        int maxSum = Integer.MIN_VALUE, currentSum = 0;

        for (int i = 0; i < array.length; i++) {
            currentSum += array[i];    // Step 1: Accumulate the current element into our local window.

            // Step 2: If the local window is the best we've seen so far, update the global record.
            if (currentSum > maxSum)    maxSum = currentSum;

            // Step 3: THE RESET. If the window becomes a net-negative, it cannot contribute
            // positively to future elements. We discard it and reset the search.
            if (currentSum < 0)        currentSum = 0;
        }
        return maxSum;
    }

    public static int Approach2(int[] array) {
        int length = array.length;
        int maxSum = Integer.MIN_VALUE;

        for (int i = 0; i < length; i++) {
            int currentSum = 0;
            for (int j = i; j < length; j++) {
                currentSum += array[j];
                if (currentSum > maxSum)
                    maxSum = currentSum;
                if (currentSum < 0)
                    currentSum = 0;
            }
        }
        return maxSum;
    }
}
```

Array = [-2, -3, 4, -1, -2, 1, 5, -3]									
Index	Array								
i=0	-2	-3	4	-1	-2	1	5	-3	
i=1	-2	-3	4	-1	-2	1	5	-3	
i=2	-2	-3	4	-1	-2	1	5	-3	
i=3	-2	-3	4	-1	-2	1	5	-3	
i=4	-2	-3	4	-1	-2	1	5	-3	
i=5	-2	-3	4	-1	-2	1	5	-3	
i=6	-2	-3	4	-1	-2	1	5	-3	
i=7	-2	-3	4	-1	-2	1	5	-3	

Maximum Sum Subarray (Kadane's Algorithm)



```
}

public static void main(String[] args) {
    int[] array = {-10, -3000, -4, 500, 4, 5, 6, 7, 8, -9000, 4, 900, -200, 800, 200, 30, -50, -70};
    long startTime = System.nanoTime();
    int result = max__Pair_Sum(array);
    long endTime = System.nanoTime();

    int result1 = Approach2(array);
    long endTime1 = System.nanoTime();
    System.out.println("Result via O(N) method is : " + result + " -> Time taken by algorithm is " + (endTime - startTime) + " ns");
    System.out.println("Result via O(N^2) method is : " + result1 + " -> Time taken by algorithm is " + (endTime1 - endTime) + " ns");
}
}
```

OUTPUT:

Result via O(N) method is : 1734 -> Time taken by algorithm is 2900 ns

Result via O(N^2) method is : 1734 -> Time taken by algorithm is 8100 ns

Notes if Any :

5.3 Trapping Rainwater Problem

[← Back to Index](#)

Problem Statement

Given an array representing the height of a bars . Calculate how much amount of it can trap after raining.

Approach / Intuition

ORIGINAL APPROACH: Pre-calculating Left and Right Max

* Logic: Create two arrays to store the maximum height seen so far from the left and right.

* The Logic: For any bar at index i , the water level is:

* $water[i] = \min(maxLeft, maxRight) - height[i]$

* Time Complexity: $O(N)$ - Three linear passes. Space Complexity: $O(N)$ - Two extra arrays of size N .

Code — Trapping Rainwater Problem

```
import static java.lang.System.nanoTime;
import static java.lang.System.out;

public class Rainwater_Trapping_Problem {
    public static int rainwater(int[] array) {
        int length = array.length;
        if (length == 0) return 0;

        int leftMax = array[0], rightMax = array[length - 1];
        int[] left = new int[length], right = new int[length];

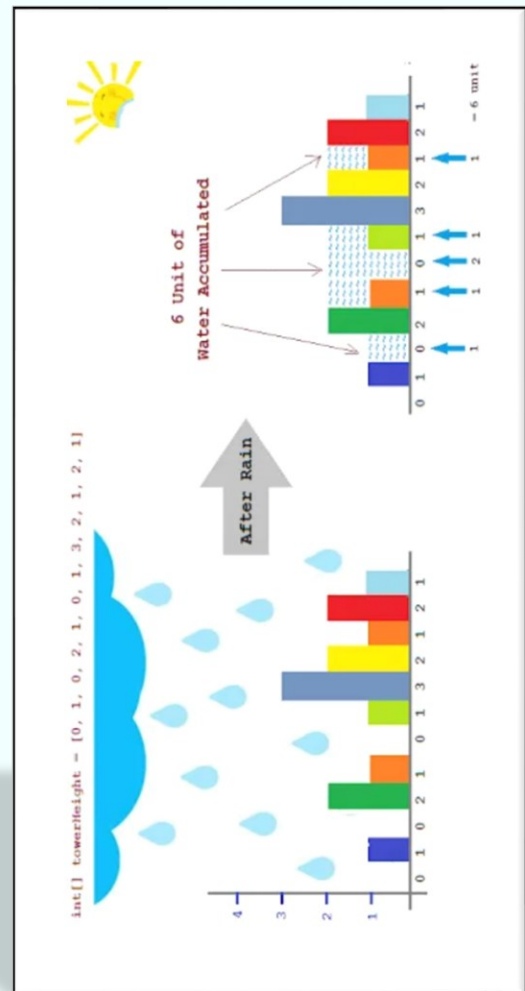
        // Fill Left Max array
        left[0] = array[0];
        for (int i = 1; i < length; i++) {
            leftMax = Math.max(leftMax, array[i]);
            left[i] = leftMax;
        }

        // Fill Right Max array
        right[length - 1] = array[length - 1];
        for (int i = length - 2; i >= 0; i--) {
            rightMax = Math.max(rightMax, array[i]);
            right[i] = rightMax;
        }

        int sum = 0;
        for (int i = 0; i < length; i++) {
            // Water trapped = min(highest left, highest right) - current height
            sum += Math.min(left[i], right[i]) - array[i];
        }

        return sum;
    }
}

/** OPTIMIZED APPROACH: Two Pointers
 * Logic: Instead of arrays, use two pointers (left and right). Move the pointer
```



```

* pointing to the smaller height, as that's the "bottleneck" for water trapping.
** Time Complexity: O(N) - Single pass. * Space Complexity: O(1) - No extra arrays used.
public static int rainwaterOptimized(int[] array) {
    int n = array.length;
    if (n <= 2) return 0; // Need at least 3 bars to trap water

    int left = 0, right = n - 1;
    int leftMax = 0, rightMax = 0;
    int totalWater = 0;

    while (left < right) {
        if (array[left] < array[right]) {
            // If current bar is taller than leftMax, update it
            if (array[left] >= leftMax) {
                leftMax = array[left];
            } else {
                // Otherwise, we can trap water
                totalWater += leftMax - array[left];
            }
            left++;
        } else {
            if (array[right] >= rightMax) {
                rightMax = array[right];
            } else {
                totalWater += rightMax - array[right];
            }
            right--;
        }
    }
    return totalWater;
}

public static void main(String[] args) {
    int[] array = {3, 1, 2, 4, 0, 1, 3, 13, 1, 2};
    // Test Original
    long start1 = nanoTime();
    int res1 = rainwater(array);
    long end1 = nanoTime();
    // Test Optimized
    long start2 = nanoTime();
    int res2 = rainwaterOptimized(array);
    long end2 = nanoTime();
    out.println("Original Result T.C O(N) , S.C (O(N)) : " + res1 + " | Time: " + (end1 - start1) + " ns");
    out.println("Optimized Result T.C O(N) , S.C (O(1)) : " + res2 + " | Time: " + (end2 - start2) + " ns");
}
}

```

OUTPUT:

Original Result T.C O(N) , S.C (O(N)) : 12 | Time: 48000 ns

Optimized Result T.C O(N) , S.C (O(1)) : 12 | Time: 2500 ns

5.4 Best Time to Buy and Sell Stock

[← Back to Index](#)

Problem Statement

Find Maximum profit possible by buying a stock and then selling it .

Approach / Intuition

APPROACH 3: One-Pass (Optimal)

Logic: Iterate through the prices while keeping track of the lowest price seen so far. For every new price, calculate the potential profit if we sold today using that minimum price.

Time Complexity: $O(N)$ - We only scan the array once. Space Complexity: $O(1)$ - We only store two integer variables.

Code — Best Time to Buy and Sell Stock

```
public class StockAndBuy {
```

```
// APPROACH 1: Brute Force Logic: Check every possible combination of buy and sell days.
```

```
    public static int maxProfitBruteForce(int[] prices) {
        int maxProfit = 0;
        for (int i = 0; i < prices.length; i++) {
            // 'j' starts from 'i + 1' because you must buy before you can sell
            for (int j = i + 1; j < prices.length; j++) {
                int profit = prices[j] - prices[i];
                if (profit > maxProfit) maxProfit = profit;
            }
        }
        return maxProfit;
    }
```

```
// APPROACH 2: Auxiliary Space (Suffix Max)
```

Logic: Pre-calculate the maximum price that occurs in the future for any given day. By storing the "best possible sell price" for each day in a separate array, we can find the profit in a single second pass.

```
    public static int maxProfitWithSpace(int[] prices) {
        int n = prices.length;
        if (n < 2) return 0;

        // Create an array where maxFuturePrices[i] is the highest price from day 'i' to the end
        int[] maxFuturePrices = new int[n];
        maxFuturePrices[n - 1] = prices[n - 1];

        // Fill the array from right to left
        for (int i = n - 2; i >= 0; i--)
            maxFuturePrices[i] = Math.max(prices[i], maxFuturePrices[i + 1]);

        int maxProfit = 0;
        // The profit for day 'i' is the (max future price) - (price on day 'i')
        for (int i = 0; i < n; i++) {
            maxProfit = Math.max(maxProfit, maxFuturePrices[i] - prices[i]);
        }
    }
```



```

    return maxProfit;
}

/** APPROACH 3: One-Pass (Optimal)
 * Logic: Iterate through the prices while keeping track of the lowest price
 * seen so far. For every new price, calculate the potential profit if we
 * sold today using that minimum price.
 * * Time Complexity: O(N) - We only scan the array once.
 * * Space Complexity: O(1) - We only store two integer variables.
 */
public static int maxProfitOptimized(int[] prices) {
    if(prices == null || prices.length == 0) return 0;
    int minPrice = Integer.MAX_VALUE; // Track the cheapest day to buy
    int maxProfit = 0;                // Track the best profit found

    for (int price : prices) {
        if(price < minPrice) {
            // Found a new historical low; update buy price
            minPrice = price;
        } else if (price - minPrice > maxProfit) {
            // If selling today yields a better profit than before, update it
            maxProfit = price - minPrice;
        }
    }
    return maxProfit;
}

public static void main(String[] args) {
    int[] array = {1, 3, 2, 4, 6, 8, 9, 11, 9, 4, 126};
    long startTime = nanoTime();
    int result = maxProfitBruteForce(array);
    long endTime = nanoTime();
    long startTime2 = nanoTime();
    int result2 = maxProfitOptimized(array);
    long endTime2 = nanoTime();
    long startTime3 = nanoTime();
    int result3 = maxProfitWithSpace(array);
    long endTime3 = nanoTime();
    out.println("--- Results ---");
    out.println("Brute Force T.C : O(N^2) & S.C : O(1) is : " + result + " (" + (endTime - startTime) + " ns)");
    out.println("Optimized T.C : O(N) & S.C : O(N) is : " + result2 + " (" + (endTime2 - startTime2) + " ns)");
    out.println("Suffix Max T.C : O(N) & S.C : O(1) is : " + result3 + " (" + (endTime3 - startTime3) + " ns)");
}
}

```

OUTPUT:

Brute Force T.C : O(N^2) & S.C : O(1) is : 125 (4600 ns)

Optimized T.C : O(N) & S.C : O(N) is : 125(1900 ns)

Suffix Max T.C : O(N) & S.C : O(1) is : 125 (39000 ns)

6.1 Bubble Sort

[← Back to Index](#)

Problem Statement

1. Bubble Sort: "Neighbor-to-Neighbor"

Idea: Adjacent comparison. Largest element “bubbles” to the end.

Approach / Intuition

BUBBLE SORT LOGIC

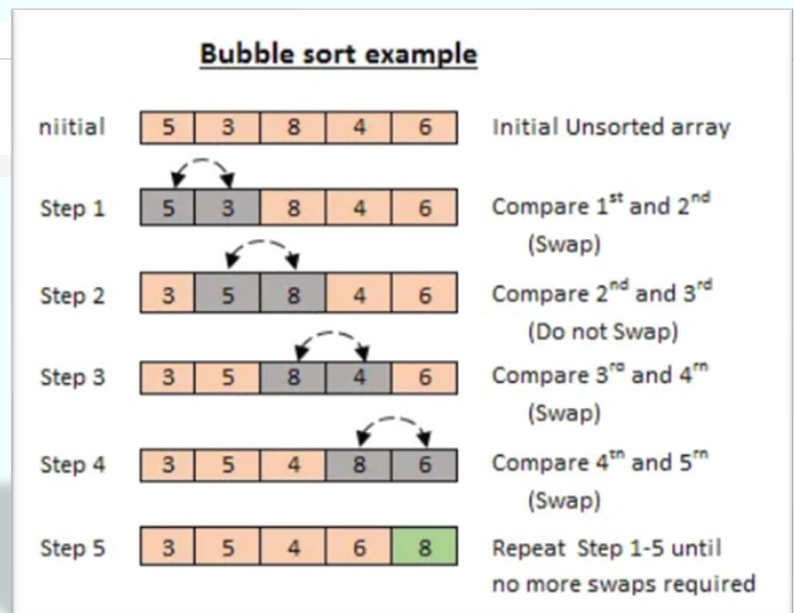
1. *The Comparison:* Start at the beginning of the array. Compare the current element (A) with its immediate neighbor (B).
 2. *The Swap:* If $A > B$, swap them. This ensures the larger value "bubbles" toward the right.
 3. *Iteration:* Move to the next pair (B & C) and repeat comparison until the end of the unsorted portion of the array is reached.
 4. *The Pass:* After one full pass, largest element of the unsorted section is guaranteed to be in its final sorted position at end.
 5. *Repeat:* Repeat process for remaining unsorted elements. With each subsequent pass, range of comparison decreases by one.
 6. *Optimization (Optional):* If a full pass occurs without any swaps, the array is already sorted, algorithm can terminate early.
- T.C Best $O(N)$, worst case $O(N^2)$ S.C $O(1)$

Code — Bubble Sort

```
import java.util.Arrays;

import static java.lang.System.nanoTime;
import static java.lang.System.out;
public class S1_BubbleSort {
    public static int[] bubbleSort(int[] array) {
        int length = array.length;
        for (int i = 0; i < length - 1; i++) {
            boolean swapped = false;

            for (int j = 0; j < length - i - 1; j++) {
                if (array[j] > array[j + 1]) {
                    /**
                     * ARCHITECTURAL NOTE ON FUNCTION CALLS:
                     * Theoretically, a function call adds overhead (stack pushing , context switching , adding new memory ,
                     * RAM etc. ). However, the modern JVM's JIT (Just-In-Time) Compiler performs "Method Inlining" injecting
                     * the swap logic directly into the loop (into the called function) to eliminate call overhead
                     * at runtime. Calling a different function inside a function is just for humans
                     * readability . It doesn't affect program overhead via context switching etc.
                     */
                    swapNumbers(array, j, j + 1);
                    swapped = true;
                }
            }
        }
    }
}
```



```

        // EARLY EXIT OPTIMIZATION: If no elements were swapped during
        // a full pass, meaning the array is already sorted.
        if (!swapped) break;
    }
    return array;
}

public static void swapNumbers(int[] array, int i, int j) {
    int temp = array[i];
    array[i] = array[j];
    array[j] = temp;
}

public static void main(String[] args) {

    int size = 5000;
    int[] array = new int[size];

    // Fill the array with random numbers.
    for (int i = 0; i < size; i++) {
        int randomNumber = (int) (Math.random() * size);
        array[i] = randomNumber;
    }

    out.println("Starting Bubble Sort for " + size + " elements...");

    long startFunction = nanoTime();
    int[] result = bubbleSort(array);
    long endFunction = nanoTime();

    out.println("Result: " + Arrays.toString(result));
    out.println("-> Time taken: " + (endFunction - startFunction) + " ns");
    out.println("-> Time in seconds: " + (double) (endFunction - startFunction) / 1_000_000_000 + " s");
}
}

```

Notes if Any :

Problem Statement

2. Insertion Sort: "Shift and Slide"

Idea: Insert element into correct position in sorted section.

Approach / Intuition

LOGIC for Insertion Sort

Think of the array as being split into two parts: a sorted part on the left and an unsorted part on the right.

- 1. Initially, the "sorted" part is just the first element array[0], because a single number is always sorted by itself.*
- 2. We pick the first element from the unsorted part (the "Key").*
- 3. We look at the sorted part to its left and shift every number that is larger than our "Key" one position to the right.*
- 4. This creates an empty slot where the "Key" finally fits.*
- 5. The "Sorted Wall" moves one step to the right, and we repeat*

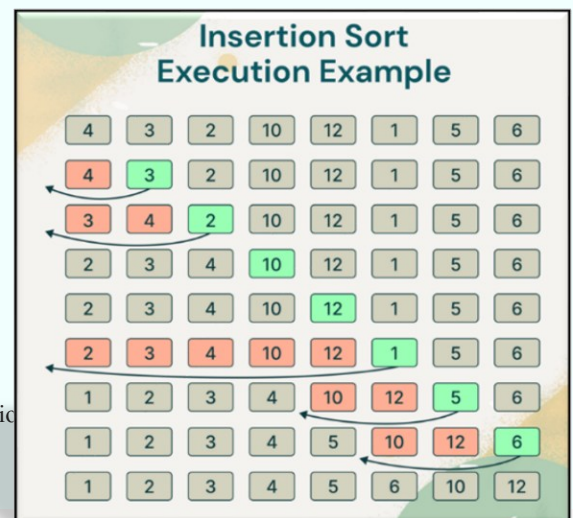
T.C: Best $O(N)$, Average $O(N^2)$, and Worst $O(N^2)$ S.C: $O(1)$ - Sorting is performed in-place without auxiliary memory.

Code — Insertion Sort

```
public class S2_InsertionSort {
    public static int[] insertionSort(int[] array) {
        int length = array.length;
        for (int i = 1; i < length; i++) { // Start from the second element (index 1)
            int key = array[i]; // 1. Pick the element to be inserted
            int j = i - 1; // 2. Start comparing with the element immediately to the left

            while (j >= 0 && array[j] > key) { // 3. The "Shifting" Phase
                array[j + 1] = array[j]; // Move the larger element to the right
                j = j - 1; // Move the pointer further left
            }
            // 4. The "Insertion" Phase
            array[j + 1] = key; // Drop the key into its correct hole
        }
        return array;
    }

    public static void main(String[] args) {
        int[] array = {1, 4, 3, 2, -1, 4, 77, 89, 54, 23, 78, 13};
        long startFunction = nanoTime(); int[] result = insertionSort(array);
        long endFunction = nanoTime();
        out.println("Result for Insertion Sort : " + Arrays.toString(result));
        out.println("-> Time taken by algorithm is: " + (endFunction - startFunction));
    }
}
```



Problem Statement

3. Selection Sort: "Search and Swap"

Idea: Find minimum → swap once → grow sorted section.

Approach / Intuition

Selection Sort is an in-place comparison-based algorithm that conceptually divides the array into two parts:

1. A sorted sublist (built from left to right)
2. An unsorted sublist (the remainder of the array).

In each iteration, the algorithm scans the unsorted sublist to find the minimum element and swaps it with the first element of that unsorted section. This "selects" the correct element for the current position and expands the sorted boundary by one.

**** Time Complexity:** - Best, Average, and Worst Case: $O(N^2)$

**** Space Complexity:** $O(1)$

Code — Selection Sort

```
import java.util.Arrays;
import static java.lang.System.out;
public class S3_SelectionSort {
    public static int[] selectionSort(int[] array) {
        int n = array.length;
        // The outer loop moves the boundary between the sorted and unsorted parts. Everything to the left of 'i' is already sorted.
        for (int i = 0; i < n - 1; i++) {
            // STEP 1: Assume the first element of the unsorted part is the smallest. We store its INDEX, not its value, so we can swap
            int minIndex = i;
            // STEP 2: The inner loop scans the remaining unsorted elements (from i + 1 to the end) to find the actual minimum value.
            for (int j = i + 1; j < n; j++) {
                if (array[j] < array[minIndex]) {
                    // We found a new "champion" smallest value
                    minIndex = j;
                }
            }
            int temp = array[minIndex];
            array[minIndex] = array[i];
            array[i] = temp;

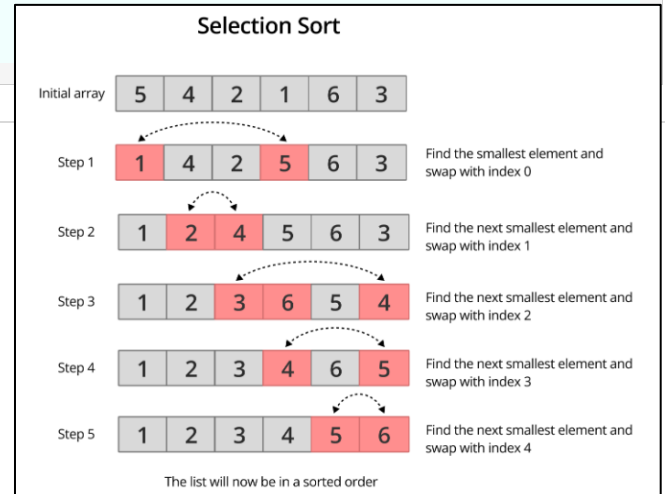
            // LOGIC CHECK: After this swap, the element at index 'i' is officially sorted. The loop then moves to 'i + 1' to find the next smallest.
        }
        return array;
    }

    public static void main(String[] args) {
        int[] data = {29, 10, 14, 37, 13, 12, 3, 34, 456, 5, 4, 6, 56, 7, 6, 8, 78, 456, 79, 456, 89, 890, 456, 909, 0, 9, 0, 9, 0, 8, 9, 788, 7,
```

456, 67, 8, 6, 7, 5, 7, 56, 6, 546, 56, 5, 6, 56, 5, 6, 5, 6, 5, 6, 5, 6, 56, 456, 56, 5, 6, 56, 5, 6, 5, 6, 5, 6, 5, 66, 7, 67, 8, 78, 6, 3, 4, 2, 3, 13, 23, 45656767, 867, 89, 78890, 879, 89};

```
out.println("\nOriginal: " + Arrays.toString(data));
long start = System.nanoTime();
selectionSort(data);
long end = System.nanoTime();
out.println("Sorted: " + Arrays.toString(data) + "\n->Time taken to Execute is " + (end - start) + " ns");
}
```

Notes if Any :



6.4 Merge Sort

[← Back to Index](#)

Problem Statement

B. Divide and Conquer Sorts ($O(N \log N)$)

These algorithms use **recursion** to break problems into smaller parts.

4. Merge Sort: "Split and Zip"

Idea: Split $\rightarrow$ recursively $\rightarrow$ merge sorted arrays.

Approach / Intuition

```
* ===== MERGE SORT (MENTAL MODEL) =====  
* Think in 2 phases:  
* 1) GO DOWN (Divide phase) * - Keep splitting array into halves Stop when size becomes 1 (already sorted)  
* 2) COME UP (Merge phase) * - Now start merging small sorted arrays - Build bigger sorted arrays step by step  
* IMPORTANT RULE: * merge() is called ONLY when both left and right are already sorted  
* =====  
* Time Complexity:  $O(N \log N)$  * Space Complexity:  $O(N)$ 
```

Code — Merge Sort

```
public class S4_MergeSort {  
    public static int[] mergeSort(int[] array) {  
        if (array.length <= 1) return array; // BASE CASE: If array has 1 element  $\rightarrow$   
        // already sorted  $\rightarrow$  return  
  
        int mid = array.length / 2; //  $\diamond$  STEP 1, DIVIDE : Split array into two halves  
        int[] leftArray = Arrays.copyOfRange(array, 0, mid), rightArray = Arrays.copyOfRange(array, mid, array.length);  
  
        /*  RECURSION FLOW (VERY IMPORTANT) * Execution does NOT go like normal top-to-bottom. It goes like:  
        * 1. Go LEFT completely (until base case)  
        * 2. Then go RIGHT completely  
        * 3. THEN merge  
        * Think: "I will NOT merge until both sides are fully sorted"  
  
        //  $\diamond$  STEP 2: SORT LEFT HALF (go deep first)  
        out.println("Splitting Left Part ..... " + Arrays.toString(leftArray));  
        mergeSort(leftArray);  
        out.println("All Left Array is being split up : " + Arrays.toString(leftArray));  
  
        //  $\diamond$  STEP 2: SORT RIGHT HALF  
        out.println("Splitting Right Part ..... " + Arrays.toString(rightArray));  
        mergeSort(rightArray);  
        out.println("All Right Array is being split up : " + Arrays.toString(rightArray));  
  
        //  $\diamond$  STEP 3: MERGE (happens while returning back)  
        // At this point:  
        // leftArray is sorted  $\checkmark$   
        // rightArray is sorted  $\checkmark$ 
```

```

    out.println("Now merging the splitted Array");
    merge(array, leftArray, rightArray);
    out.println("\n\n");

    return array;
}

/** ===== MERGE FUNCTION =====
 * Input:
 * left[] → already sorted
 * right[] → already sorted
 * Output:
 * result[] → combined sorted array
 * Idea:
 * Compare the smallest elements from both arrays Pick the smaller one and move forward
 */

private static void merge(int[] result, int[] leftArray, int[] rightArray) {

    int leftIndex = 0; // pointer for left array
    int rightIndex = 0; // pointer for right array
    int k = 0; // pointer for result array

    // ◇ Compare elements from both arrays
    while (leftIndex < leftArray.length && rightIndex < rightArray.length) {
        if (leftArray[leftIndex] <= rightArray[rightIndex])
            result[k++] = leftArray[leftIndex++]; // take from left
        else
            result[k++] = rightArray[rightIndex++]; // take from right
    }

    // ◇ Copy remaining elements (if any)
    while (leftIndex < leftArray.length) // If left still has elements
        result[k++] = leftArray[leftIndex++];
    while (rightIndex < rightArray.length) // If right still has elements
        result[k++] = rightArray[rightIndex++];

    out.println("===== Merge Result ===== " + Arrays.toString(result));
}

public static void main(String[] args) {
    int[] data = {1, 29, 10, 14, 37, 13, 12, 3, 34, 456, 5, 4, 6, 56, 7, 6, 8, 78};
    out.println("\nOriginal: " + Arrays.toString(data));
    long start = System.nanoTime();
    mergeSort(data);
    long end = System.nanoTime();
    out.println("Sorted: " + Arrays.toString(data));
    out.println("Time taken: " + (end - start) + " ns");
}
}

```

🌳 1. DIVIDE PHASE (Top → Down)

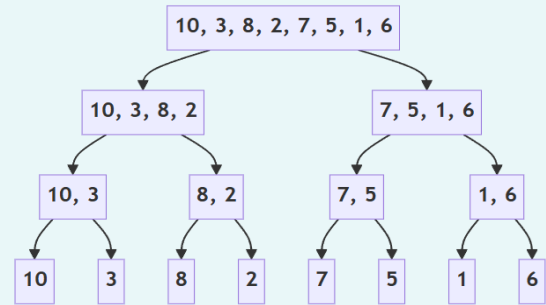
👉 Merge sort keeps splitting until single elements

```
mergeSort([21 items])
├── Left (10 items)
│   └── [21,4,18,7,2,15,9,1,13,6]
└── Right (11 items)
    └── [11,3,20,8,5,14,10,19,12,16,17]
```

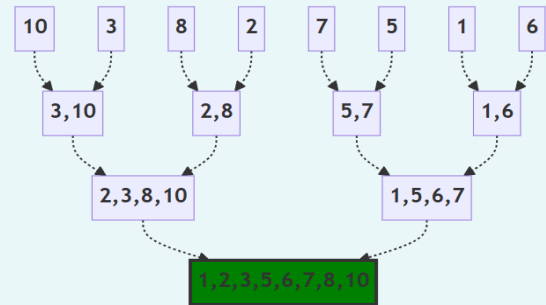
📌 Keep going LEFT first (depth-first)

```
[21,4,18,7,2,15,9,1,13,6]
├── [21,4,18,7,2]
│   ├── [21,4]
│   │   ├── [21]
│   │   └── [4]
│   │   └── merge → [4,21]
│   ├── [18,7,2]
│   │   ├── [18]
│   │   └── [7,2]
│   │       ├── [7]
│   │       └── [2]
│   │       └── merge → [2,7]
│   │       └── merge → [2,7,18]
│   └── merge → [2,4,7,18,21]
├── [15,9,1,13,6]
├── [15,9]
│   ├── [15]
│   └── [9]
│   └── merge → [9,15]
├── [1,13,6]
├── [1]
├── [13,6]
├── [13]
├── [6]
├── merge → [6,13]
├── merge → [1,6,13]
├── merge → [1,6,9,13,15]
└── FINAL LEFT MERGE:
    [2,4,7,18,21] + [1,6,9,13,15]
    → [1,2,4,6,7,9,13,15,18,21]
```

Divide Phase



Merge Phase



📌 NOW RIGHT SIDE (only AFTER left is done)

```
[11,3,20,8,5,14,10,19,12,16,17]
├── [11,3,20,8,5]
│   ├── [11,3]
│   │   └── [3,11]
│   ├── [20,8,5]
│   │   ├── [20]
│   │   └── [8,5]
│   │       └── [5,8]
│   │       └── merge → [5,8,20]
│   └── merge → [3,5,8,11,20]
├── [14,10,19,12,16,17]
│   ├── [14,10,19]
│   │   └── [10,14,19]
│   ├── [12,16,17]
│   │   └── [12,16,17]
│   └── merge → [10,12,14,16,17,19]
└── FINAL RIGHT MERGE:
    [3,5,8,11,20] + [10,12,14,16,17,19]
    → [3,5,8,10,11,12,14,16,17,19,20]

🔥 FINAL MERGE (ROOT)
LEFT:
[1,2,4,6,7,9,13,15,18,21]
RIGHT:
[3,5,8,10,11,12,14,16,17,19,20]
```

👉 Merge step:

```
→ [1,2,3,4,5,6,7,8,9,10,11,12,
→ 13,14,15,16,17,18,19,20,21]
```

Problem Statement

5. Quick Sort: "Pivot Partition"

Idea: Choose pivot → partition → recurse.

Approach / Intuition

Quick Sort is a Divide & Conquer algorithm that picks an element as a 'pivot' & partitions the given array around picked pivot.

Logic: * 1. Pivot Selection: Pick an element (commonly the last element).

* 2. Partitioning: Rearrange array so that all elements smaller than pivot are on left, & all elements larger are on right.

* 3. Recursion: Recursively apply the same logic to the left and right sub-arrays.

*** Time Complexity: * - Best & Average Case: $O(N \log N)$ * Worst Case: $O(N^2)$ - *** Space Complexity: * - $O(\log N)$

Code — Quick Sort

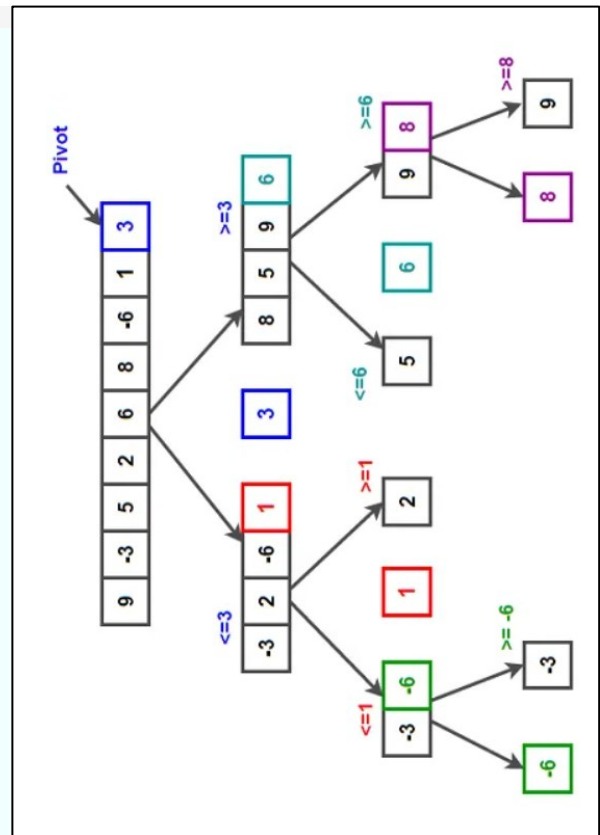
```
import java.util.Arrays;
import static java.lang.System.out;

public class S5_QuickSort {
    public static int[] quickSort(int[] array) {
        if (array == null || array.length == 0)
            return array;
        sort(array, 0, array.length - 1);
        return array;
    }

    private static void sort(int[] array, int low, int high) {
        if (low < high) {
            int pivot = partition(array, low, high);
            sort(array, low, pivot - 1); // Left side
            sort(array, pivot + 1, high); // Right side
        }
    }

    private static int partition(int[] array, int low, int high) {
        int pivot = array[high]; // We choose the last element as the pivot
        int i = (low - 1);

        for (int j = low; j < high; j++) {
            if (array[j] < pivot) {
                i++;
                int temp = array[i];
```



```

        array[i] = array[j];
        array[j] = temp;
    }
}

// Final Swap: Place the pivot in its correct sorted position by swapping it with the element at i + 1.
int temp = array[i + 1];
array[i + 1] = array[high];
array[high] = temp;

return i + 1; // Return the position of the pivot
}

public static void main(String[] args) {
    int[] data = {29, 10, 14, 37, 13, 12, 3, 34, 456, 5, 4, 6, 56, 7, 6, 8, 78, 456, 79, 456, 89, 890, 456
        , 909, 0, 9, 0, 9, 0, 8, 9, 788, 7, 456, 67, 8, 6, 7, 5, 7, 56, 6, 546, 56, 5, 6, 56, 5, 6, 5, 6, 5, 6, 56, 456, 56, 5, 6, 56, 5
        , 6, 5, 6, 5, 6, 5, 66, 7, 67, 8, 78, 6, 3, 4, 2, 3, 13, 23, 45656767, 867, 89, 78890, 879, 89};

    out.println("\nOriginal: " + Arrays.toString(data));
    long start = System.nanoTime();
    quickSort(data);
    long end = System.nanoTime();
    out.println("Sorted: " + Arrays.toString(data) + "\n->Time taken to Execute is " + (end - start) + " ns");
}
}

```

Notes if Any :

Module 07: Searching

7.1 Binary Search

[← Back to Index](#)

Problem Statement

In this , we are to search elements “key” in a sorted array

Approach / Intuition

Divide the array into 2 parts on the basis of array length.

If middle elements is greater than the key , then elements must be present in left part. So discard the right part completely and repeats above process , i.e., find middle elements then compare with key and so on...

Code — Binary Search

```
import java.io.*;
import java.util.Arrays;

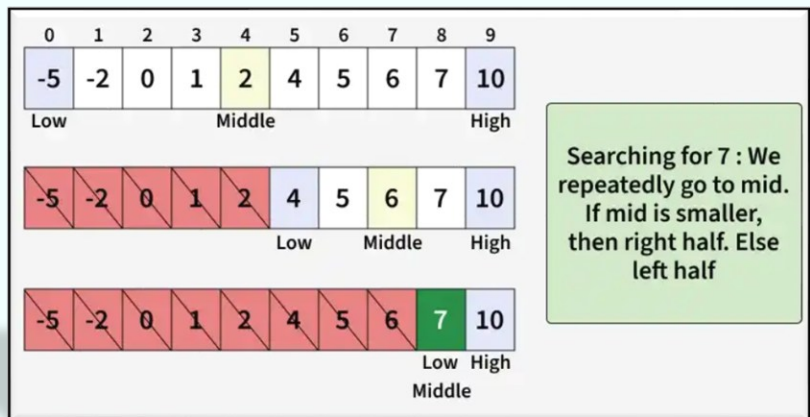
public class BinarySearch {

    public static boolean binarySearch(int[] array, int key) {

        int low = 0, high = array.length - 1;
        int operationPerformed = 0;

        while (low <= high) {
            int mid = (low + high) / 2;
            if (array[mid] == key) {
                System.out.printf("Elements Found at index %d & Total Operations performed " +
                    "to search elements %d is : %d OPERATIONS\n", mid, key, operationPerformed);
                return true;
            }
            if (key < array[mid]) {
                high = mid - 1;
                operationPerformed++;
            }
            if (key > array[mid]) {
                low = mid + 1;
                operationPerformed++;
            }
        }

        System.out.printf("Elements Not found & Total Operations performed to search " +
            "elements %d is %d OPERATIONS \n", key, operationPerformed);
    }
}
```



```

        return false;

    }

    public static boolean recursionBinarySearch(int[] array, int key, int low, int high,
                                                int totalOperations) {
//    Base case , Elements not found , returning false
        if (low > high) {
            return false;
        }
        int mid = (high + low) / 2;
        if (array[mid] == key) {
            System.out.printf("Elements Found at index %d via Recursion & Total operations " +
                              "taken to search element %d is : %d OPERATIONS \n", mid, key,
                              totalOperations);
            return true;
        }
        if (array[mid] > key)
            return recursionBinarySearch(array, key, low, (mid - 1), totalOperations + 1);
        if (array[mid] < key)
            return recursionBinarySearch(array, key, (mid + 1), high, totalOperations + 1);
        System.out.println("Elements Not Found ");
        return false;
    }

    public static void main(String[] args) throws IOException {
        int[] array = {0, 2, 4, 5, 5, 8, 9, 10, 10, 11, 11, 12, 13, 14, 15, 15, 17, 18, 18, 19, 19, 20, 22, 22, 24, 24, 24, 24, 26, 32, 37, 38, 38,
            39, 39, 40, 4999};

        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        int key;

        do {
            System.out.println(Arrays.toString(array));
            System.out.print("Enter elements to Search from above list ( -1 to QUIT ) : -> ");
            String input = bufferedReader.readLine();
            key = Integer.parseInt(input);
            long startFunction = System.nanoTime();
            binarySearch(array, key);
            long endFunction = System.nanoTime();

            recursionBinarySearch(array, key, 0, array.length - 1, 0);
            long endRecursion = System.nanoTime();
            System.out.printf("Time taken to execute function via Normal Binary Search : %d ns " +
                              "\n", (endFunction - startFunction));
            System.out.printf("Time taken to execute function via Recursion : %d ns \n", (endRecursion - endFunction));

        } while (key != -1);
    }
}

```

7.2 Allocate Minimum No. of Pages (Google Interview)

[← Back to Index](#)

Problem Statement

* *GOOGLE INTERVIEW QUESTION*

Given an array of pages where `array[i]` represents number of pages in *i*-th book, and '*k*' students, allocate books such that:

- * 1. Each student gets at least one book.
- * 2. Each student reads a contiguous sequence of books.
- * 3. The maximum number of pages assigned to any single student is *MINIMIZED*.

Approach / Intuition

Core Logic

- **Search Space:** The answer must lie between the maximum pages in a single book (low) and the sum of all pages (high).
- **Binary Search:** Check the middle value (mid). If it is possible to allocate books such that no student reads more than mid pages, look for a smaller maximum (high = mid - 1). Otherwise, increase the limit (low = mid + 1).
- **Feasibility Check:** Iterate through the books and greedily assign them to students without exceeding mid pages per student. [1, 2, 3, 4, 5]

Code — Allocate Minimum Number of Pages (Binary Search on Answer)

```
public class BookAllocation {

    public int minPages(int[] array, int totalStudents) {
        // Base case: If books are fewer than students, allocation isn't possible
        // under the "at least one book per student" rule.
        if (array.length < totalStudents) return -1;

        int min = maxOf(array); // A student must be able to hold at least the largest book
        int max = sumOf(array); // Max limit is one student reading everything
        int res = -1;

        while (min <= max) {
            int mid = min + (max - min) / 2; // Prevents potential overflow , same as int mid = ( low + high ) / 2

            if (isFeasible(array, totalStudents, mid)) {
                res = mid; // This capacity works, let's try to find a smaller one
                max = mid - 1;
            } else {
                min = mid + 1; // This capacity is too small, increase the limit
            }
        }
        return res;
    }

    /**
     * Helper to check if it's possible to distribute books such that
     * no student reads more than 'limit' pages.
     */
    private boolean isFeasible(int[] array, int k, int limit) {
        int students = 1, currentSum = 0;
```

```

    for (int pages : array) {
        if (currentSum + pages > limit) {
            students++;
            currentSum = pages;
        } else
            currentSum += pages;
    }
    return students <= k;
}

private int sumOf(int[] array) {
    int sum = 0;
    for (int i : array) sum += i;
    return sum;
}

private int maxOf(int[] array) {
    int max = 0;
    for (int i : array) if (i > max) max = i;
    return max;
}

public static void main(String[] args) {
    BookAllocation obj = new BookAllocation();

    int[] pages = {12, 34, 67, 90, 92, 94, 96, 98, 99, 100};
    int totalStudents = 6;
    int result = obj.minPages(pages, totalStudents);
    if (result != -1)
        System.out.println("The minimized maximum number of pages a student has to read is: " + result);
    else
        System.out.println("Allocation impossible: More students than books available.");
}
}

```

OUTPUT:

The minimized maximum number of pages a student has to read is: 190

Code — Linear Search

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.util.Arrays;

public class LinearSearch {

    public static int findIndex(int[] array, int search) {
        for (int i = 0; i < array.length; i++) {
            if (array[i] == search) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) throws IOException {
        int[] array = {0, 2, 4, 5, 5, 8, 9, 10, 10, 11, 11, 12, 13, 14, 15, 15, 17, 18, 18, 19, 19, 4999};
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        int number = 10;
        long start = System.nanoTime();
        int result = findIndex(array, number);
        long end = System.nanoTime();

        System.out.println("\n-----");
        if (result != -1) {
            System.out.printf("FOUND AT INDEX :%d\n", result);
        } else {
            System.out.println("NOT FOUND : Number not in list.");
        }
        System.out.printf("Time: %d ns\n", (end - start));
        System.out.println("-----");

        //      System.out.println("\nPress Enter to continue...");
        //      bufferedReader.readLine(); // Pause so the user can see the result
    }
}
```

7.4 Search in an Infinite / Unbounded Sorted Array

[← Back to Index](#)

Problem Statement

Search elements in an infinite array say lakh of crores or millions of billions of trillions of infinite array

* Here if we use concept `int length = array.length`, compiler will give error as data is infinite.

* If somehow it is possible to get array's length via `array.length`, then also don't use this.

* SOLVE WITHOUT USING `array.length` TO GET LAST ARRAY'S INDEX

USE THE UNBOUNDED BINARY SEARCH TECHNIQUE.

* $T.C = O(\log n + \log n) = O(\log n)$ * $S.C = O(1)$

Approach / Intuition

Since we cannot find the end of the array using `.length`, we must find the search boundaries dynamically in two steps: [1, 2]

1. **Exponential Search:** Find the range [low, high] where target element lies by doubling the index range step-by-step.
2. **Binary Search:** Perform a standard binary search within that discovered range.

To simulate an infinite array in Java without throwing an `ArrayIndexOutOfBoundsException`, we catch the exception and treat it as a signal that the index has exceeded the array bounds (similar to encountering an infinite value).

Code — Search in an Infinite / Unbounded Sorted Array

```
public class Search_In_Infinite_Array {
    public int searchInInfiniteArray(int[] array, int key) {
        int lowRange = 0;
        int highRange = 1;

        // Exponentially increase the range until the key is within [low, high]
        // We use a try-catch or check for potential out-of-bounds if the array isn't actually "infinite" but just very large.
        try {
            while (key > array[highRange]) {
                int newLow = highRange + 1;
                // Double the range size: (high - low + 1) * 2
                highRange = highRange + (highRange - lowRange + 1) * 2;
                lowRange = newLow;
                System.out.println("Searching elements in range : " + lowRange + " - " + highRange);
            }
        } catch (ArrayIndexOutOfBoundsException e) {
            // If we hit the end of a non-infinite array, the catch handles the boundary
            System.out.println("Array Out of Bounds Exceptions : ");
        }
        return searchKey(array, lowRange, highRange, key);
    }
    // For loop will take O(n) time so overall T.C is O(n + log n) so skipping searching
    // via for loop instead use binary searches recursion method
    // for (int i = lowRange; i <= highRange; i++) {
    //     if (array[i] == key)
    //         return i;
    // }
}
```

```

public int searchKey(int[] array, int lowRange, int highRange, int key) {
    if (lowRange > highRange)
        return -1;
    // if low and high are very large, their sum might exceed the maximum value for an integer (2^{31} -1),
    // causing an Integer Overflow. Using low + (high - low) / 2 is the safer, professional way to calculate midpoint.
    int mid = lowRange + (highRange - lowRange) / 2;
    if (key == array[mid])        return mid;
    if (key > array[mid])        return searchKey(array, (mid + 1), highRange, key);
    if (key < array[mid])        return searchKey(array, lowRange, (mid - 1), key);
    return -1;
}

public static void main(String[] args) {
    long startFunction = System.nanoTime();
    int[] array = {0, 2, 4, 5, 5, 8, 9, 10, 10, 11, 11, 12, 13, 14, 15, 15, 17, 18, 18, 19, 19, 20, 22, 22, 24, 24, 24, 24, 24, 26, 32, 37, 38, 38,
39, 39, 40, 41, 43, 48, 51, 52, 52, 53, 53, 54, 57, 58, 59, 59, 60, 60, 61, 61, 61, 61, 62, 64, 65, 66, 67, 4999};
    Search_In_Infinite_Array obj = new Search_In_Infinite_Array();
    int key = 67;
    int result = obj.searchInInfiniteArray(array, key);
    long endFunction = System.nanoTime();
    System.out.printf(result != -1 ? "Elements " + key + " found at index : " + result + " " :
        "Sorry , Elements is not present. Try searching other elements ");
    System.out.println("\n-> Time taken to execute function is : " + (endFunction - startFunction) + " ns");
}
}

```

OUTPUT:

Searching elements in range :2 - 5

Searching elements in range :6 - 13

Searching elements in range :14 - 29

Searching elements in range :30 - 61

Elements 67 found at index : 59 -> Time taken to execute function is : 4601800 ns

Notes if Any :

7.5 Search in Sorted and Rotated Array

[← Back to Index](#)

Problem Statement

* A rotated sorted array is an array that was originally sorted in ascending order but has had its elements shifted (rotated) to the left or right by some number of positions. Example:

* Original Sorted Array: [10, 20, 30, 40, 50, 60, 70] , Rotated by 3 positions: [40, 50, 60, 70, 10, 20, 30]

* In this example, the last three elements were moved to the front, or the first three were "pushed" to the back.

* Search Properties: Because the array is no longer fully sorted, a standard

* Binary Search won't work out of the box. However, because each "half" is still sorted, we can use a Modified Binary Search

Rotation Count | Resulting Array | Description

* -----|-----|-----

* 0 (Original) | [1, 2, 3, 4, 5] | No rotation; standard sorted array.

* 1 Rotation | [5, 1, 2, 3, 4] | The largest element moves to the front.

* 2 Rotations | [4, 5, 1, 2, 3] | Two elements moved from back to front.

* 4 Rotations | [2, 3, 4, 5, 1] | Almost a full cycle; smallest at the end.

* Time Complexity: $O(\log n)$ - Modified Binary Search * Space Complexity: $O(1)$ - Constant space

Approach / Intuition

Core Logic

In rotated sorted array, dividing array in half always results in **at least one half being perfectly sorted**. We use this property:

1. **Find Mid:** Calculate the middle index mid.
2. **Identify Sorted Half:**
 - If $\text{array}[\text{low}] \leq \text{array}[\text{mid}]$, the **left half** is sorted.
 - Otherwise, the **right half** is sorted.

Eliminate Half: Check if the target falls within the bounds of the sorted half. If it does, narrow the search to that half. If it does not, search the other half.

Code — Search in Sorted and Rotated Array

```
public class Search_In_Sorted_And_Rotated_Array {
    public static int rotatedArray(int[] array, int key) {
        int low = 0;
        int high = array.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2; // same as int mid = (low + high) / 2
            if (array[mid] == key) return mid;
            if (array[low] <= array[mid]) { // Scenario 1: Left part [low...mid] is sorted
                // Key lies within the sorted left half
                if (key >= array[low] && key < array[mid]) high = mid - 1;
                else low = mid + 1;
            }
            // Scenario 2: Right part [mid...high] is sorted
        }
    }
}
```

```

    else {
        // Key lies within the sorted right half
        if (key > array[mid] && key <= array[high])        low = mid + 1;
        else        high = mid - 1;
    }
}
return -1;
}

public static void main(String[] args) throws IOException {
    int[] array = {11, 12, 13, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
    while (true) {
        System.out.println("\n--- Rotated Binary Search Utility ---");
        System.out.print("Enter number to search (or -1 to QUIT): ");
        String input = bufferedReader.readLine();
        if (input == null || input.isEmpty()) continue;

        int number;
        try        number = Integer.parseInt(input);
        catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter a number.");
            continue;
        }
        if (number == -1) break;
        long start = System.nanoTime();
        int result = rotatedArray(array, number);
        long end = System.nanoTime();
        if (result != -1)        System.out.printf("SUCCESS: Found %d at index %d\n", number, result);
        else        System.out.println("NOT FOUND: Number not in list.");
        System.out.printf("Execution Time: %d ns\n", (end - start));
    }
}
}

```

OUTPUT:

Enter number to search (or -1 to QUIT): 2
 SUCCESS: Found 2 at index 4 Execution Time: 5000 ns

Enter number to search (or -1 to QUIT): 13
 SUCCESS: Found 13 at index 2 Execution Time: 1500 ns

Module 08: Hashing

8.1 Count Distinct Elements in Every Window of Size K

[← Back to Index](#)

Problem Statement

Count distinct Elements in every window of size 'k'

* Example: -> array = {1,2,-2,1,3,1,1,3} and k = 4 distinct elements are :

* for index {0-3} as k=4 , distinct elements are :3 {1,2,-2}

* for index {1-4} as k=4 , distinct elements are :4 {2,-2,1,3}

* for index {2-5} as k=4 , distinct elements are :3 {-2,1,3}

* for index {3-6} as k=4 , distinct elements are :2 {1,3}

* for index {4-7} as k=4 , distinct elements are :2 {3,1}

* T.C = $O(n * k)$

Approach / Intuition

For first k elements add all data modify HashMap only after first k elements had been inserted . Why Clearing all data from map and again insert same data . We can instead remove from extreme left by getting hashKey .

REMEMBER : Unlike array we can not get hashmaps index , because it does not store data at index logic rather it hashes the value, and then it stores data at that hashing place not at index . Again to retrieve data we get hashKey via method `map.get(array[i])` which returns data stored at that place

Code — Count Distinct Elements in Every Window of Size K

```
public class DistinctElements_PerWindow {
    public static List<Integer> distinctElements_PerWindow_TC(int[] array, int k) {
        List<Integer> list = new ArrayList<>();
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < k; i++) {
            map.put(array[i], map.getDefault(array[i], 0) + 1);
        }
        list.add(map.size());
        // Now Push & remove elements from index k+1
        for (int i = k; i < array.length; i++) {
            int in = array[i]; // to insert new elements inside HashMap
            int out = array[i - k]; // to remove elements inside HashMap from index array[i-k] i.e., from extreme left side
            // If only last data then we have to delete that data or else it will store 0 at that place which will be
            // counted as a different value and our result would be compromised
            if (map.get(out) == 1) map.remove(out);
            else map.put(out, map.getDefault(out, 0) - 1);
            map.put(in, map.getDefault(in, 0) + 1);
            list.add(map.size());
            System.out.println("For window of size " + k + " Distinct Elements are : " + map.keySet());
        }
        return list;
    }

    public static List<Integer> distinctElements_PerWindow(int[] array, int k) {
```

```

/** Time Complexity = O(n * k) */
List<Integer> list = new ArrayList<>();
HashSet<Integer> set = new HashSet<>(k);
for (int i = 0; i <= (array.length - k); i++) {
    for (int j = 0; j < k; j++) {
        set.add(array[i + j]);
    }
    list.add(set.size());
    set.clear();
}
return list;
}

public static void main(String[] args) throws IOException {
    int[] array = {1, 2, -2, 1, 3, 1, 1, 3};
    BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));
    int k;
    while (true) {
        System.out.println("\nArray Content: " + Arrays.toString(array));
        System.out.print("Enter Key size (or -1 to QUIT): ");
        String input = bufferedReader.readLine();
        if (input == null || input.isEmpty()) continue;
        try {
            k = Integer.parseInt(input);
        } catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter a k value : ");
            continue;
        }
        if (k == -1) {
            System.out.println("Goodbye!");
            break;
        }
        long start = System.nanoTime();
        List<Integer> result = distinctElements_PerWindow(array, k);
        long end = System.nanoTime();
        List<Integer> result2 = distinctElements_PerWindow_TC(array, k);
        long end2 = System.nanoTime();
        System.out.print("Total distinct Elements per Window is : " + result);
        System.out.printf(" Time: %d ns\n", (end - start));
        System.out.print("Total distinct Elements per Window T.C : " + result2);
        System.out.printf(" Time: %d ns\n", (end2 - end));
    }
}

```

OUTPUT

```

Array Content: [1, 2, -2, 1, 3, 1, 1, 3]
Enter Key size (or -1 to QUIT): 6
For window of size 6 Distinct Elements are : [1, -2, 2, 3]
For window of size 6 Distinct Elements are : [1, -2, 3]
Total distinct Elements per Window is : [4, 4, 3] Time: 22600 ns
Total distinct Elements per Window T.C : [4, 4, 3] Time: 232100 ns

Array Content: [1, 2, -2, 1, 3, 1, 1, 3]
Enter Key size (or -1 to QUIT): 7
For window of size 7 Distinct Elements are : [1, -2, 2, 3]
Total distinct Elements per Window is : [4, 4] Time: 32500 ns
Total distinct Elements per Window T.C : [4, 4] Time: 175900 ns

```

[← Back to Index](#)

75

8.3 Intersection of Two Arrays

[← Back to Index](#)

Code — Intersection of Two Arrays

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;

public class Intersection_Of_Array {
    public static int findUnion(int[] array1, int[] array2) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int value : array1)    map.put(value, map.getOrDefault(value, 0) + 1);
        for (int value : array2)    map.put(value, map.getOrDefault(value, 0) + 1);

        HashSet<Integer> set = new HashSet<>();
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            if (entry.getValue() > 1)    set.add(entry.getKey());
        }
        System.out.println("Intersection of array via HashMap is : " + set);
        return set.size();
    }

    public static int findUnion_HashSet(int[] array1, int[] array2) {
        HashSet<Integer> set = new HashSet<>();
        HashSet<Integer> intersection = new HashSet<>();
        for (int value : array1)    set.add(value);
        for (int value : array2) {
            if (set.contains(value))    intersection.add(value);
        }
        System.out.println("Intersection of array via HashSet is : " + intersection);
        return intersection.size();
    }

    static void main(String[] args) {
        int[] array1 = {1, 2, 3, 4, 5};
        int[] array2 = {11, 12, 13, 4, 5};
        int result = findUnion(array1, array2);
        int result2 = findUnion_HashSet(array1, array2);
    }
}
```

OUTPUT

```
Intersection of array via HashMap is : [4, 5]
Intersection of array via HashSet is : [4, 5]
Time taken to execute function is : 1081500 ns
Time taken to execute function is : 233600 ns
```

8.4 Subarray with Given Sum

[← Back to Index](#)

Problem Statement

To find the subArray / part of Array whose sum is equal to the target Sum. Given sub array should be sequential

Example :- for array {1,3,5,6,3,4,5} for targetSum = 14 , subarray is {5,6,3} continuously

Approach / Intuition Using HashMap

1. Traverse the array while maintaining a running sum (currentSum).
2. If currentSum equals the target sum, a valid subarray is found from index 0 to i.
3. Store each prefix sum and its index in a HashMap.
4. For each element, check if (currentSum - targetSum) exists in the HashMap.
5. If it exists, the subarray from the next index of that prefix sum to the current index has the required sum.
6. If no such subarray is found after traversing the array, report that the target sum does not exist.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Code — Subarray with Given Sum

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

public class SubArray_With_Given_Sum {
    public static List<Integer> findSubArraysSum(int[] array, int targetSum) {
        List<Integer> list = new ArrayList<>();
        int start = 0, currentSum = 0;

        for (int i = 0; i < array.length; i++) {
            int value = array[i];
            currentSum += value;
            while (currentSum > targetSum && start < i) {
                currentSum -= array[start++];
            }
            if (currentSum == targetSum) {
                for (int j = start; j <= i; j++) {
                    list.add(array[j]);
                }
                return list;
            }
        }
        System.out.println("Lists data are : " + list);
        return list;
    }

    public static void findSubArraysSum_HashMap(int[] array, int targetSum) {
```

OUTPUT

```
Target found from index via HashMap : 49 to 52
To get Target Sum = 635 , Arrays are : [534, 56, 45]
Time taken to execute functions is : 58700 ns
Time taken to execute HashMap function is : 6851100 ns
```

```

int currentSum = 0, start = 0, end = -1;
HashMap<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < array.length; i++) {
    currentSum += array[i];
    if((currentSum - targetSum) == 0) {
        start = 0;
        end = i;
        break;
    }
    if (map.containsKey(currentSum - targetSum)) {
        start = map.get(currentSum - targetSum);
        end = i;
        break;
    }
    map.put(currentSum, i);
}
if (end == -1)
    System.out.println("Target sum not found via HashMap");
else
    System.out.println("Target found from index via HashMap : " + start + " to " + end);
}

static void main(String[] args) {
    int[] array = {1, 2, 3, 4, 5, 45, 3, 5, 56, 57, 6, 78, 8, 79, 89, 8, 79, 7, 86, 7, 5
        , 6, 4, 5, 3, 4, 2, 456, 23, 4, 3, 5, 54, 6, 5, 7, 67, 86789, 788090, 9, -9, 0, -89, 678, 54,
        6, 34534, 45, 542354, 34, 534, 56, 45, 56, 7,};
    int targetSum = 635;
    long startTime = System.nanoTime();
    List<Integer> list = findSubArraysSum(array, targetSum);
    long endTime = System.nanoTime();
    findSubArraysSum_HashMap(array, targetSum);
    long endTime2 = System.nanoTime();
    System.out.println("To get Target Sum = " + targetSum + " , Arrays are : " + list.toString());
    System.out.println("Time taken to execute functions is : " + (endTime - startTime) + " ns");
    System.out.println("Time taken to execute HashMap function is : " + (endTime2 - endTime) + " ns");
}
}

```

Notes if Any :

Code — Union of Two Arrays

```
import com.sun.source.tree.BreakTree;
import java.util.HashSet;

public class Union_Arrays {
    public static HashSet<Integer> findUnion(int[] array1, int[] array2) {
        HashSet<Integer> set = new HashSet<>();
        for (int value1 : array1)
            set.add(value1);
        for (int value2 : array2)
            set.add(value2);
        return set;
    }

    static void main(String[] args) {
        int[] array1 = {1, 2, 3, 4, 5};
        int[] array2 = {11, 12, 13, 14, 51};
        System.out.println("Union are :" + findUnion(array1, array2));
    }
}
```

OUTPUT:

Union are :[1, 2, 3, 51, 4, 5, 11, 12, 13, 14]

Notes if Any :

For Detailed Notes on Hashing (HashMap & HashSet) , please visit <https://github.com/manishrnl/DSA-Practice-Problem/blob/main/08%20Hashing/Hashing-report.md>

Module 09: Heap (Priority Queue)

9.1 Min Heap Implementation

[← Back to Index](#)

Problem Statement

Implement **Insertion** and **Deletion** operations in a **Min Heap** using an array representation.

A **Min Heap** is a complete binary tree where the value of every parent node is smaller than or equal to its child nodes.

In Min Heap, Parent node must always be smaller than child nodes . Array representation formulas:

- $\text{Parent} = (i - 1) / 2$
- $\text{Left Child} = 2 * i + 1$
- $\text{Right Child} = 2 * i + 2$

The program should:

1. Insert a new element into the Min Heap while maintaining the heap property.
2. Delete a given element from the Min Heap and restore the heap structure after deletion.

Approach / Intuition

1. Insertion in Min Heap : Idea

When inserting a new element:

1. Add the element at the last position of the heap.
2. Compare it with its parent.
3. If the inserted element is smaller than its parent, swap them.
4. Continue moving upward until the Min Heap property is satisfied.

This process is called **Heapify Up** (or Bubble Up).

Steps

1. Create a new array with one extra space.
2. Copy existing heap elements.
3. Insert the new value at the last index.
4. Find its parent using $(\text{childIndex} - 1) / 2$.
5. While the child is smaller than the parent:
 - Swap child and parent.
 - Move to the parent's index.
6. Stop when the heap property is restored.

2. Deletion from Min Heap : Idea -To delete an element:

1. Find the element's index.
2. Replace it with the last element of the heap.
3. Remove the last element.
4. Restore the heap property by moving the replaced node downward.

This process is called **Heapify Down** (or Bubble Down).

Steps

1. Search for the value to delete.
2. Store its index.
3. Replace it with the last heap element.
4. Reduce heap size by one.
5. Compare the replaced node with its children.
6. Swap with the smaller child if required.
7. Continue until the Min Heap property is restored.

Code — Min Heap Implementation

```
public class MinHeap {
    public static void insertInto_MinHeap(int[] array, int value) {
        int[] newArray = new int[array.length + 1]; // Create new array with one extra space
        System.arraycopy(array, 0, newArray, 0, array.length); // Copy original heap
        newArray[array.length] = value; // Insert new value at LAST position
        System.out.println("Before Heapify:" + Arrays.toString(newArray));
        int childIndex = newArray.length - 1; // Child index = last index

        while (childIndex > 0) {
            int parentIndex = (childIndex - 1) / 2; // Find parent
            if (newArray[childIndex] < newArray[parentIndex]) { // Min Heap condition
                int temp = newArray[childIndex];
                newArray[childIndex] = newArray[parentIndex];
                newArray[parentIndex] = temp;

                childIndex = parentIndex; // Move upward
                System.out.println("After swaps :" + Arrays.toString(newArray));
            } else break;
        }
        System.out.println("Heap after insertion:" + Arrays.toString(newArray));
    }

    public static void deleteFrom_MinHeap(int[] array, int value) {
        System.out.println("\n#####");
        System.out.println("Performing Delete Operation");
        int length = array.length;
        int deletedIndex = -1;
        // Step 1: Find element
        for (int i = 0; i < length; i++) {
            if (array[i] == value) {
                deletedIndex = i;
                break;
            }
        }
        // Value not found
        if (deletedIndex == -1) {
            System.out.println("Value not found.");
            return;
        }

        // Step 2: Replace deleted node with last node
        array[deletedIndex] = array[length - 1];
        // Step 3: Remove last element
        int[] newArray = Arrays.copyOf(array, length - 1);
        System.out.println("Before Heapify:");
        System.out.println(Arrays.toString(newArray));
        int current = deletedIndex;

        // Step 4: Heapify Down
        while (true) {
            int leftChild = 2 * current + 1;
```

OUTPUT

```
Before Heapify:[10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 35]
After swaps :[10, 20, 30, 40, 35, 60, 70, 80, 90, 100, 50]
Heap after insertion:[10, 20, 30, 40, 35, 60, 70, 80, 90, 100, 50]

#####
Performing Delete Operation
Before Heapify:
[10, 20, 30, 100, 40, 50, 60, 70, 80, 90]
After Heapify:
[10, 20, 30, 70, 40, 50, 60, 100, 80, 90]
```

```

    int rightChild = 2 * current + 2;
    int smallest = current;
    // Compare left child
    if (leftChild < newArray.length && newArray[leftChild] < newArray[smallest])    smallest = leftChild;
    // Compare right child
    if (rightChild < newArray.length && newArray[rightChild] < newArray[smallest])    smallest = rightChild;
    // Heap property satisfied
    if (smallest == current)    break;
    // Swap
    int temp = newArray[current];
    newArray[current] = newArray[smallest];
    newArray[smallest] = temp;

    // Move downward
    current = smallest;
}
System.out.println("After Heapify "+ Arrays.toString(newArray));
}

public static void main(String[] args) {
    int[] array = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int value = 35;
    insertInto_MinHeap(array, value);
    deleteFrom_MinHeap(new int[] {10, 20, 30, 35, 40, 50, 60, 70, 80, 90, 100}, value);
}
}

```

Operation	Technique Used	Time Complexity	Space Complexity
Insert	Heapify Up	$O(\log n)$	$O(n)$
Delete	Search + Heapify Down	$O(n)$	$O(n)$

Notes if Any :

Problem Statement

Implement **Insertion** and **Deletion** operations in a **Max Heap** using an array representation.

A **Max Heap** is a complete binary tree where the value of every parent node is greater than or equal to its child nodes.

Array representation formulas (0-based indexing):

- $\text{Parent} = (i - 1) / 2$
- $\text{Left Child} = 2 * i + 1$
- $\text{Right Child} = 2 * i + 2$

The program should:

1. Insert a new element into the Max Heap while maintaining the heap property.
2. Delete a given element from the Max Heap and restore the heap structure after deletion.

Approach / Intuition

1. Insertion in Max Heap . Idea : When inserting a new element:

1. Place the new element at the last position of the heap.
2. Compare it with its parent.
3. If the inserted element is greater than its parent, swap them.
4. Continue moving upward until the Max Heap property is satisfied.

This process is called **Heapify Up** (or Bubble Up).

Steps

1. Create a new array with one extra space.
2. Copy all existing heap elements into the new array.
3. Insert the new value at the last index.
4. Find its parent using $(\text{childIndex} - 1) / 2$.
5. While the child is greater than the parent:
 - Swap parent and child.
 - Move to the parent's position.
6. Stop when the heap property is restored.

2. Deletion from Max Heap. Idea: To delete an element:

1. Find the index of the element to be deleted.
2. Replace it with the last element of the heap.
3. Remove the last element.
4. Restore the heap property by moving the replaced node downward.

This process is called **Heapify Down** (or Bubble Down).

Steps

1. Search for the value to delete.
2. Store its index.
3. Replace it with the last heap element.
4. Reduce heap size by one.
5. Compare the replaced node with its children.
6. Swap with the larger child if necessary.
7. Continue until the Max Heap property is restored.

Code — Max Heap Implementation

```
public class MaxHeap {
    public static void insertInto_MaxHeap(int[] array, int value) {
        int[] newArray = new int[array.length + 1]; // Step 1: Create new array with extra space
        System.arraycopy(array, 0, newArray, 0, array.length);
        newArray[array.length] = value; // Step 2: Insert new value at last position
        int childIndex = newArray.length - 1; // Child index = last index
        System.out.println("Original Arrays are : " + Arrays.toString(newArray));

        while (childIndex > 0) { // Step 3: Heapify Up
            int parentIndex = (childIndex - 1) / 2; // Find parent index
            if (newArray[childIndex] > newArray[parentIndex]) { // If child > parent → swap
                int temp = newArray[childIndex];
                newArray[childIndex] = newArray[parentIndex];
                newArray[parentIndex] = temp;
                childIndex = parentIndex; // Move upward
            } else {
                // Heap property satisfied
                break;
            }
            System.out.println("After Swaps : " + Arrays.toString(newArray));
        }
        System.out.println("Heap after insertion:" + Arrays.toString(newArray));
    }

    public static void deleteFrom_MaxHeap(int[] array, int value) {
        System.out.println("\n#####\nPerforming Delete operations for value : " + value);
        int n = array.length;
        int deleteIndex = -1; // Step 1: Find index of value which needs to be deleted
        for (int i = 0; i < n; i++) {
            if (array[i] == value) {
                deleteIndex = i;
                break;
            }
        }
        if (deleteIndex == -1) { // Value not found
            System.out.println("Value not found in heap , retry with valid data ...");
            return;
        }
        array[deleteIndex] = array[n - 1]; // Step 2: Replace delete element with last element
        int[] newHeap = Arrays.copyOf(array, n - 1); // Step 3: Create new array excluding last element
        System.out.println("Before Heapify:" + Arrays.toString(newHeap));
        int currentIndex = deleteIndex;

        while (true) { // Step 4: Heapify Down
            int leftChild = (2 * currentIndex + 1), rightChild = (2 * currentIndex + 2), largest = currentIndex;
            if (leftChild < newHeap.length && newHeap[leftChild] > newHeap[largest]) // Compare left child
                largest = leftChild;
            if (rightChild < newHeap.length && newHeap[rightChild] > newHeap[largest]) // Compare right child
                largest = rightChild;
            if (largest == currentIndex) // Heap property satisfied
                break;
            // Swap current node with largest child
            int temp = newHeap[currentIndex];
            newHeap[currentIndex] = newHeap[largest];
            newHeap[largest] = temp;
            currentIndex = largest;
        }
        System.out.println("After Delete:" + Arrays.toString(newHeap));
    }
}
```

```

        break;
        int temp = newHeap[currentIndex];
        newHeap[currentIndex] = newHeap[largest];
        newHeap[largest] = temp;
        currentIndex = largest; // Move downward
    }
    System.out.println("Heap after deletion:" + Arrays.toString(newHeap));
}

public static void main(String[] args) {
    int[] array = {90, 70, 60, 40, 50, 20, 10};
    int value = 80;
    insertInto_MaxHeap(array, value);
    deleteFrom_MaxHeap(new int[] {90, 80, 70, 60, 40, 50, 20, 10}, value);
}
}

```

Operation	Technique Used	Time Complexity	Space Complexity
Insert	Heapify Up	$O(\log n)$	$O(n)$
Delete	Search + Heapify Down	$O(n)$	$O(n)$

Notes if Any :

For Detailed Notes on Heap , please visit <https://github.com/manishrnl/DSA-Practice-Problem/blob/main/09%20Heap%20Data%20Structures/README.md>

Module 10 :

10.1 Sub-Module Name

[← Back to Index](#)

Problem Statement

```
while (childIndex > 0) { // Step 3: Heapify Up
```

Approach / Intuition

```
while (childIndex > 0) { // Step 3: Heapify Up
```

Code — Heap Theory (complete binary tree, heap order property)

```
public class Solution {  
  
}
```

Notes if Any :
